

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
TRƯỜNG ĐẠI HỌC THỦY LỢI



HỒ THỦY TIÊN

KẾT HỢP THUẬT TOÁN A* CẢI TIẾN VÀ
THUẬT TOÁN THAM LÂM ĐỂ LẬP KẾ HOẠCH ĐƯỜNG ĐI
CỦA ROBOT DI ĐỘNG ĐA MỤC TIÊU

ĐỒ ÁN TỐT NGHIỆP

HÀ NỘI, NĂM 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
TRƯỜNG ĐẠI HỌC THỦY LỢI

HỒ THỦY TIÊN

**KẾT HỢP THUẬT TOÁN A* CẢI TIẾN VÀ
THUẬT TOÁN THAM LÂM ĐỂ LẬP KẾ HOẠCH ĐƯỜNG ĐI
CỦA ROBOT DI ĐỘNG ĐA MỤC TIÊU**

Ngành : Hệ thống thông tin

Mã số: 7480104

NGƯỜI HƯỚNG DẪN ThS. Trần Thị Cẩm Giang

HÀ NỘI, NĂM 2025

GÁY BÌA ĐỒ ÁN TỐT NGHIỆP

HỒ THỦY TIÊN

ĐỒ ÁN TỐT NGHIỆP

HÀ NỘI, NĂM 2025



CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập - Tự do - Hạnh phúc



NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ tên sinh viên: Hồ Thủy Tiên

Hệ đào tạo: Đại học chính quy

Lớp: 63 HTTT 2

Ngành: Hệ thống thông tin

Khoa: Công nghệ thông tin

TÊN ĐỀ TÀI: KẾT HỢP THUẬT TOÁN A* CẢI TIẾN VÀ THUẬT TOÁN THAM LAM ĐỂ LẬP KẾ HOẠCH ĐƯỜNG ĐI CỦA ROBOT DI ĐỘNG ĐA MỤC TIÊU

1. CÁC TÀI LIỆU CƠ BẢN:

- [1] D. L. H. O. J. & H. D. Xiang, "Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot," *Scientific reports*, vol. 12.1, p. 13273, 2022.
- [2] N. Y. Y. & A. M. A. Adzhar, "A review on autonomous mobile robot path planning algorithms.," *Advances in Science, Technology and Engineering Systems Journal*, vol. 5(3), pp. 236-240, 2020.
- [3] P. H. & N. D. T. Luân, "Nghiên cứu ứng dụng thuật toán Aco (ANT COLONY OPTIMIZATION) tối ưu thời gian và chi phí cho dự án xây dựng," *Tạp chí Phát triển KH&CN*, 2010.
- [4] A. V. S. M. & S. A. Rezvanian, "An overview of ant colony optimization algorithms for dynamic optimization problems," *Optimization Algorithms- Classics and Recent Advances*, 2023.

2. NỘI DUNG CÁC PHẦN THUYẾT MINH VÀ TÍNH TOÁN: Tỷ lệ %

Nội dung các phần	Tỷ lệ %
Chương 1: Tổng quan và cơ sở lý thuyết	30%
Chương 2: Bài toán lập kế hoạch đường đi cho robot di động đa mục tiêu, đa đích đến	20%
Chương 3: Giải thuật đề xuất	20%
Chương 4: Cài đặt thực nghiệm	20%

3. GIÁO VIÊN HƯỚNG DẪN TỪNG PHẦN

Phần	Họ và tên giáo viên hướng dẫn
Chương 1: Tổng quan và cơ sở lý thuyết	ThS. Trần Thị Cẩm Giang
Chương 2: Bài toán lập kế hoạch đường đi cho robot di động đa mục tiêu, đa đích đến	ThS. Trần Thị Cẩm Giang
Chương 3: Giải thuật đề xuất	ThS. Trần Thị Cẩm Giang
Chương 4: Cài đặt thực nghiệm	ThS. Trần Thị Cẩm Giang

4. NGÀY GIAO NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Ngày tháng năm 2025

Trưởng Bộ môn
(Ký và ghi rõ Họ tên)

Giáo viên hướng dẫn chính
(Ký và ghi rõ Họ tên)

Nhiệm vụ Đồ án tốt nghiệp đã được Hội đồng thi tốt nghiệp của Khoa thông qua.

Ngày..... tháng năm 2025

Chủ tịch Hội đồng
(Ký và ghi rõ Họ tên)

Sinh viên đã hoàn thành và nộp bản Đồ án tốt nghiệp cho Hội đồng thi ngày..... tháng... năm 2025

Sinh viên làm Đồ án tốt nghiệp
(Ký và ghi rõ Họ tên)



TRƯỜNG ĐẠI HỌC THỦY LỢI
KHOA CÔNG NGHỆ THÔNG TIN

BẢN TÓM TẮT ĐỀ CƯƠNG ĐỒ ÁN TỐT NGHIỆP

Tên đề tài: Kết hợp thuật toán A* cải tiến và thuật toán tham lam để lập kế hoạch đường đi của robot di động đa mục tiêu

Sinh viên thực hiện: Hồ Thủy Tiên

Lớp: 63HTTT2

Mã sinh viên: 2151163732

Số điện thoại: 0345454920

Email: thuytienql0912@gmail.com

Giáo viên hướng dẫn: ThS. Trần Thị Cẩm Giang

TÓM TẮT ĐỀ TÀI

Trong bối cảnh tự động hóa và công nghiệp thông minh đang ngày càng phát triển, robot di động đóng vai trò then chốt trong các lĩnh vực như: kho vận, nhà máy thông minh và hệ thống giao hàng tự động. Những robot này thường phải thực hiện nhiệm vụ di chuyển đến đích trong môi trường có nhiều chướng ngại vật. Tuy nhiên, nếu chỉ đơn thuần hướng đến việc hoàn thành lộ trình mà không tối ưu hóa đường đi, robot sẽ dễ gặp phải các vấn đề như tốn thời gian, tiêu hao năng lượng, tăng nguy cơ va chạm và giảm hiệu quả vận hành.

Đề tài này tập trung nghiên cứu và mô phỏng bài toán lập kế hoạch đường đi tối ưu cho robot di động trong môi trường có vật cản, bằng cách kết hợp giữa thuật toán A* cải tiến và thuật toán tham lam. Thuật toán A* cải tiến giúp đảm bảo tìm ra lộ trình ngắn và an toàn hơn, trong khi thuật toán tham lam được tích hợp để tăng tốc quá trình lựa chọn hướng đi dựa trên tiêu chí ưu tiên như chi phí ước lượng hoặc khoảng cách đến đích. Sự kết hợp này góp phần nâng cao hiệu quả lập kế hoạch đường đi, giúp cải thiện đáng kể về thời gian xử lý, chiều dài đường đi và độ trơn mượt của quỹ đạo.

Mô hình có thể được ứng dụng trong hệ thống robot vận chuyển trong kho hàng, dây chuyền sản xuất tự động hoặc các hệ thống dẫn đường trong môi trường công nghiệp thông minh – nơi yêu cầu đường đi tối ưu, an toàn và hiệu quả.

MỤC TIÊU CHÍNH

- Nghiên cứu bài toán lập kế hoạch đường đi tối ưu cho robot di động trong môi trường có chướng ngại vật.
- Hiểu rõ đặc trưng của môi trường hoạt động như: cấu trúc bản đồ, vị trí chướng ngại vật, và giới hạn vận hành của robot.
- Xây dựng và triển khai thuật toán kết hợp A* cải tiến với thuật toán tham lam để nâng cao hiệu suất tìm đường.
- Đánh giá hiệu quả của mô hình đề xuất thông qua các tiêu chí: thời gian lập kế hoạch, độ dài đường đi, góc quay và mức độ trơn mượt của quỹ đạo.
- So sánh kết quả với các thuật toán truyền thống nhằm xác định tính ưu việt của mô hình đề xuất.

KẾT QUẢ DỰ KIẾN

- Xây dựng thành công mô hình giúp robot di động có khả năng lập kế hoạch đường đi một cách tối ưu trong môi trường phức tạp.
 - Mô hình kết hợp A* cải tiến và thuật toán tham lam kỳ vọng cải thiện đáng kể các yếu tố: thời gian lập kế hoạch, tổng chiều dài đường đi, số lần rẽ, góc quay gấp nhằm góp phần nâng cao hiệu suất, độ trơn mượt và ổn định khi di chuyển.
 - Mô hình có khả năng ứng dụng trong các hệ thống tự động hóa vận chuyển tại kho hàng, nhà máy và các môi trường sản xuất thực tế.
- Công nghệ sử dụng: Python với thư viện Numpy, Matplotlib, NetworkX.

LỜI CAM ĐOAN

Em xin cam đoan đây là đồ án tốt nghiệp của bản thân em. Các kết quả trong đồ án tốt nghiệp này là trung thực, và không sao chép từ bất kỳ một nguồn nào và dưới bất kỳ hình thức nào. Việc tham khảo các nguồn tài liệu (nếu có) đã được thực hiện trích dẫn và ghi nguồn tài liệu tham khảo đúng quy định.

Tác giả ĐATN

Hồ Thủy Tiên

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn đến Trường Đại học Thủy Lợi, nơi đã tạo điều kiện học tập, rèn luyện và trưởng thành cho em trong suốt những năm tháng đại học. Những kiến thức và kỹ năng được trang bị tại đây đã trở thành nền tảng vững chắc cho sự phát triển chuyên môn của em.

Đặc biệt, em xin gửi lời cảm ơn chân thành tới cô Trần Thị Cẩm Giang – giảng viên hướng dẫn. Trong suốt quá trình thực hiện đồ án tốt nghiệp, cô đã tận tình định hướng, góp ý sâu sắc và truyền đạt những kiến thức quý báu giúp em hoàn thiện từng phần của đồ án một cách có hệ thống. Em thật sự trân trọng và biết ơn sự tận tâm, trách nhiệm và tình cảm mà cô đã dành cho em trong suốt thời gian qua.

Bên cạnh đó, em cũng xin gửi lời cảm ơn đến các thầy cô trong Khoa và Nhà trường, những người đã không ngừng truyền cảm hứng và kiến thức cho em trong suốt quá trình học tập.

Em cũng không quên gửi lời cảm ơn chân thành đến gia đình – những người luôn là điểm tựa vững chắc, đồng hành và động viên em bằng tất cả tình yêu thương vô điều kiện. Và cuối cùng, xin cảm ơn các bạn bè, anh chị đồng môn đã luôn bên cạnh chia sẻ kiến thức, kinh nghiệm và cùng nhau vượt qua quãng đường đầy thử thách của đời sinh viên.

Dù kết quả đồ án còn những hạn chế nhất định, nhưng em luôn tâm niệm đây là thành quả của một quá trình học tập nghiêm túc, nỗ lực không ngừng và là nền tảng quan trọng cho con đường phía trước.

Em xin trân trọng cảm ơn!

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
MỤC LỤC	iii
DANH MỤC HÌNH ẢNH	v
DANH MỤC BẢNG BIỂU	vi
DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ	vii
MỞ ĐẦU	1
CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT	4
1.1 Tổng quan về robot di động	4
1.1.1 Khái niệm	4
1.1.2 Phân loại	4
1.1.3 Ứng dụng	5
1.1.4 Thách thức	8
1.2 Tổng quan về lập kế hoạch đường đi cho robot	9
1.2.1 Định nghĩa và phân loại	9
1.2.2 Các phương pháp mô hình hóa môi trường	10
1.3 Thuật toán A^*	15
1.3.1 Tổng quan thuật toán A^*	15
1.3.2 Nguyên lý hoạt động	16
1.3.3 Ưu và nhược điểm	17
1.3.4 Ứng dụng trong lập kế hoạch đường đi	18
1.4 Thuật toán tham lam	18
1.4.1 Tổng quan thuật toán	18
1.4.2 Nguyên lý hoạt động	19
1.4.3 Ưu và nhược điểm	20
1.4.4 Ứng dụng của thuật toán	21
1.5 Thuật toán tối ưu hóa đàn kiến	21
1.5.1 Tổng quan	21
1.5.2 Nguyên lý hoạt động	24
1.5.3 Ưu và nhược điểm	26
1.5.4 Ứng dụng của thuật toán	27
CHƯƠNG 2: BÀI TOÁN LẬP KẾ HOẠCH ĐƯỜNG ĐI CHO ROBOT DI ĐỘNG	

ĐA MỤC TIÊU, ĐA ĐÍCH ĐẾN	29
2.1 Phát biểu bài toán	29
2.2 Mô hình hóa bài toán	30
2.3 Các nghiên cứu liên quan	31
CHƯƠNG 3. GIẢI THUẬT ĐỀ XUẤT	33
3.1 Thuật toán A* cải tiến kết hợp ACO (ImpA*ACO)	33
3.1.1 Lý do đề xuất kết hợp ImpA* với ACO	33
3.1.2 Thuật toán A* cải tiến	34
3.1.3 Thuật toán kết hợp A* cải tiến và ACO	41
3.1.4 Ưu và nhược điểm	43
CHƯƠNG 4. CÀI ĐẶT THỰC NGHIỆM	45
4.1 Cài đặt môi trường	45
4.2 Các tiêu chí đánh giá	46
4.3 Kịch bản thực nghiệm	47
4.4 Kết quả thực nghiệm	49
4.4.1 Case 1: So sánh toàn diện giữa các thuật toán	49
4.4.2 Case 2: Ảnh hưởng của mật độ chướng ngại vật	54
4.4.3 Case 3: Ảnh hưởng của số lượng đích đến	59
4.4.4 Case 4: Ảnh hưởng của hình dạng chướng ngại vật	63
4.4.5 Case 5: Đánh giá dựa trên mô phỏng môi trường thực tế	67
KẾT LUẬN	71
TÀI LIỆU THAM KHẢO	73

DANH MỤC HÌNH ẢNH

Hình 1.1. Robot được sử dụng trong vận chuyển hàng hóa trong kho.	6
Hình 1.2. Robot giao hàng tự động được triển khai tại bệnh viện ở Singapore.	6
Hình 1.3. Robot hút bụi, lau sàn.	7
Hình 1.4. Robot Spirit được thiết kế để khám phá địa hình giống như trên mặt trăng. ..	7
Hình 1.5. Robot hình người thực hiện nhiệm vụ cứu hộ con người trong thảm họa.	8
Hình 1.6. Robot giáo dục SoftBank Pepper.	8
Hình 1.7. Phân loại lập kế hoạch đường đi.	10
Hình 1.8. Đồ thị hiển thị (Visibility Graph)	12
Hình 1.9. Đồ thị Voronoi (Voronoi Graph)	12
Hình 1.10. Đồ thị tiếp tuyến (Tangent Graph)	13
Hình 1.11. Phương pháp tiếp cận dựa trên cây tứ phân	14
Hình 1.12. Minh họa hàm đánh giá $f(n)$ trong thuật toán A^*	16
Hình 1.13. Minh họa đường đi được tạo bởi thuật toán A^* trên bản đồ dạng lưới	17
Hình 1.14. Mô phỏng hành vi tìm kiếm thức ăn của bầy kiến trong tự nhiên.	22
Hình 1.15. Cơ chế lựa chọn và củng cố đường đi tối ưu trong thuật toán ACO.	24
Hình 1.16. Lưu đồ hoạt động của thuật toán ACO	25
Hình 2.1. Minh họa môi trường lưới 10×10 với nhiều đích đến và chướng ngại vật. .	30
Hình 3.1. Minh họa tối ưu hóa làm mịn đường đi	39
Hình 3.2. Lưu đồ thuật toán A^* cải tiến kết hợp ACO	42
Hình 4.1. Kết quả thực nghiệm case 1	51
Hình 4.2. Kết quả thực nghiệm case 2	56
Hình 4.3. Kết quả thực nghiệm case 3	60
Hình 4.4. Kết quả thực nghiệm case 4	65
Hình 4.5. Bộ dữ liệu văn phòng.	68
Hình 4.6. Bộ dữ liệu nhà kho.	68
Hình 4.7. Bộ dữ liệu thành phố	69

DANH MỤC BẢNG BIỂU

Bảng 3.1. Bảng quy tắc 3 hướng	36
Bảng 3.2. Bảng quy tắc 5 hướng	36
Bảng 4.1 Chú thích giao diện khi chạy thực nghiệm	49
Bảng 4.2 Kết quả thực nghiệm case 1	51
Bảng 4.3. Kết quả thực nghiệm case 2	56
Bảng 4.4. Kết quả thực nghiệm case 3	60
Bảng 4.5. Kết quả thực nghiệm case 4	65
Bảng 4.6. Kết quả thực nghiệm case 5	69

DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ

Từ viết tắt	Thuật ngữ	Giải thích
A*	Astar	Thuật toán tìm đường heuristic kết hợp chi phí thực và chi phí ước lượng
ACO	Ant Colony Optimization	Thuật toán tối ưu hóa dựa trên hành vi tìm đường của đàn kiến
AI	Artificial Intelligence	Trí tuệ nhân tạo – lĩnh vực nghiên cứu mô phỏng trí tuệ con người bằng máy tính
AUVs	Autonomous Underwater Vehicles	Phương tiện dưới nước tự hành, thường dùng cho khảo sát hoặc nhiệm vụ dưới biển
BFS	Breadth-First Search	Thuật toán tìm kiếm theo chiều rộng, mở rộng các nút theo từng lớp
DFS	Depth-First Search	Thuật toán tìm kiếm theo chiều sâu, đi sâu theo nhánh trước khi quay lại
ImpA*	Improved Astar	Phiên bản cải tiến của thuật toán A* với hàm đánh giá và hướng mở rộng thông minh
ImpA*ACO	Improved Astar with Ant Colony Optimization	Phiên bản kết hợp giữa A* cải tiến và thuật toán tối ưu hóa đàn kiến
ImpA*G	Improved Astar with Greedy	Phiên bản cải tiến của A* kết hợp chiến lược tham lam để xử lý bài toán đa mục tiêu
IWD	Intelligent Water Drops	Thuật toán tối ưu lấy cảm hứng từ dòng chảy nước thông minh trong tự nhiên, dùng trong bài toán tìm đường.
PSO	Particle Swarm Optimization	Thuật toán bầy đàn hạt, mô phỏng hành vi của đàn chim hoặc cá để tìm lời giải tối ưu

MỞ ĐẦU

1. Bối cảnh

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, tự động hóa và trí tuệ nhân tạo đã trở thành động lực cốt lõi cho sự phát triển của nhiều ngành công nghiệp. Robot di động được coi là một công nghệ đột phá, đóng vai trò quan trọng trong quá trình hiện đại hóa và được ứng dụng rộng rãi trong các lĩnh vực như nhà kho thông minh, logistics, nhà máy, bệnh viện và giao thông vận tải. Một trong những vấn đề cốt lõi trong việc triển khai robot di động chính là khả năng tự động lên kế hoạch đường đi tối ưu khi di chuyển trong môi trường phức tạp, đặc biệt là với nhiều mục tiêu di chuyển. Lập kế hoạch đường đi hiệu quả không chỉ giúp robot tiết kiệm thời gian và năng lượng mà còn đảm bảo an toàn và tính khả thi khi vận hành thực tế. Do đó, việc nghiên cứu các phương pháp tối ưu đường đi cho robot di động đa mục tiêu là một trong những chủ đề thu hút sự quan tâm trong những năm gần đây.

2. Lý do chọn đề tài

Trong lĩnh vực lập kế hoạch đường đi cho robot di động, nhiều thuật toán kinh điển đã được nghiên cứu và ứng dụng như Dijkstra, Breadth-First Search (BFS), Depth-First Search (DFS) và đặc biệt là A* truyền thống, mặc dù phổ biến, vẫn còn tồn tại các hạn chế như đường đi tạo ra thường có nhiều điểm rẽ không cần thiết, các khúc cua gấp và không mượt mà, dẫn đến việc di chuyển không ổn định và tốn nhiều năng lượng. Hơn nữa, việc áp dụng các thuật toán này vào bài toán đa mục tiêu một cách hiệu quả vẫn là một thách thức. Nhận thấy những hạn chế này, việc nghiên cứu và phát triển các giải pháp lập kế hoạch đường đi thông minh hơn, linh hoạt hơn là một hướng đi cần thiết. Chính vì lý do trên, đề tài này tập trung nghiên cứu và phát triển giải pháp lập kế hoạch đường đi cho robot di động bằng cách kết hợp giữa A* cải tiến và thuật toán tham lam, nhằm nâng cao chất lượng đường đi cũng như khả năng thích ứng trong môi trường có nhiều mục tiêu. Ngoài ra, để tăng tính khách quan trong đánh giá, đề án cũng thử nghiệm thêm phương pháp A* cải tiến kết hợp thuật toán ACO như một hướng tham khảo nâng cao.

3. Mục tiêu nghiên cứu

- Nghiên cứu cơ sở lý thuyết và triển khai thuật toán A^* cải tiến làm nền tảng, có khả năng tạo ra đường đi tối ưu và mượt mà hơn so với A^* truyền thống.
- Triển khai giải pháp nền tảng kết hợp A^* cải tiến và thuật toán tham lam để giải quyết bài toán đa mục tiêu, phân tích ưu điểm và nhược điểm về tính tối ưu của phương pháp này.
- Đề xuất và triển khai giải pháp mở rộng kết hợp A^* cải tiến và thuật toán tối ưu hóa đàn kiến, nhằm khắc phục hạn chế của thuật toán tham lam và tìm kiếm một lộ trình di chuyển có chất lượng cao hơn.
- Xây dựng ứng dụng mô phỏng để trực quan hóa, thu thập dữ liệu và so sánh hiệu năng của các giải pháp đã triển khai dựa trên các tiêu chí cụ thể (độ dài, góc cua, thời gian xử lý).

4. Nhiệm vụ nghiên cứu

- Nghiên cứu lý thuyết: tổng quan tài liệu, các bài báo khoa học liên quan đến lập kế hoạch đường đi, thuật toán A^* , thuật toán tham lam, và thuật toán tối ưu hóa đàn kiến.
- Thiết kế và triển khai các thuật toán: A^* truyền thống, A^* cải tiến, giải pháp kết hợp với thuật toán tham lam và giải pháp kết hợp với thuật toán tối ưu hóa đàn kiến.
- Triển khai mô phỏng thực nghiệm và đánh giá hiệu quả.
- Tổng hợp và viết báo cáo: trình bày toàn bộ quá trình nghiên cứu, triển khai và kết quả đạt được trong báo cáo đồ án tốt nghiệp.

5. Đối tượng và phạm vi nghiên cứu

- Đối tượng nghiên cứu: các thuật toán lập kế hoạch đường đi, bao gồm A^* , A^* cải tiến, thuật toán tham lam, và thuật toán tối ưu hóa đàn kiến.
- Phạm vi nghiên cứu:

- Môi trường: không gian 2D, được mô hình hóa dưới dạng bản đồ lưới với các chướng ngại vật tĩnh.
- Robot: được xem là một robot dạng điểm, có khả năng di chuyển đa hướng và bỏ qua các ràng buộc về động học, động lực học thực tế.
- Bài toán: lập kế hoạch đường đi toàn cục, nơi toàn bộ thông tin bản đồ và vị trí các mục tiêu được biết trước.

6. Phương pháp nghiên cứu

- Phương pháp nghiên cứu lý thuyết: phân tích, tổng hợp kiến thức từ các tài liệu, giáo trình, bài báo khoa học.
- Phương pháp thực nghiệm: triển khai các thuật toán, xây dựng chương trình mô phỏng để chạy thực nghiệm và thu thập dữ liệu.
- Phương pháp so sánh, phân tích: đối chiếu, so sánh các kết quả thực nghiệm dựa trên các tiêu chí đã đề ra để đánh giá và rút ra kết luận.

7. Bố cục của đề án

Chương 1: Tổng quan và cơ sở lý thuyết.

Chương 2: Bài toán lập kế hoạch đường đi cho robot di động đa mục tiêu, đa đích đến.

Chương 3: Giải thuật đề xuất.

Chương 4: Cài đặt thực nghiệm.

CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về robot di động

1.1.1 Khái niệm

Robot di động (Mobile Robot) là một hệ thống cơ điện tử tự hành, có khả năng di chuyển trong không gian làm việc mà không bị cố định tại một vị trí cụ thể. Chúng sử dụng các cảm biến và công nghệ liên quan để nhận diện môi trường xung quanh và ra quyết định di chuyển phù hợp mà không cần sự can thiệp trực tiếp của con người. Những robot này vận hành dựa trên sự kết hợp giữa trí tuệ nhân tạo (AI) và các thành phần cơ học, cho phép chúng thích ứng với môi trường thực tế và thực hiện nhiều loại nhiệm vụ khác nhau. Điểm khác biệt chính của robot di động so với các loại robot khác là khả năng di chuyển một cách tự chủ (autonomous mobility). Những robot này được lập trình để ra quyết định dựa trên các thuật toán, cho phép chúng có khả năng thu nhận và diễn giải dữ liệu môi trường, phân tích, tự học hỏi từ dữ liệu đó để tối ưu hóa hành vi và từ đó đưa ra các hành động phù hợp trong thời gian thực [1].

1.1.2 Phân loại

Robot di động có thể được phân loại theo hai tiêu chí phổ biến [2]:

- Dựa trên môi trường hoạt động.
- Dựa trên cơ cấu vận hành.

a. Phân loại dựa trên môi trường hoạt động

Robot di động được phân loại dựa trên môi trường làm việc như sau:

- Robot trên không: đây là các robot có khả năng di chuyển trong không trung, thường dùng trong giám sát, giao hàng hoặc chụp ảnh trên cao. Ví dụ như máy bay không người lái (UAVs) hoặc drone.
- Robot trên mặt đất: đây là loại hình robot di động phổ biến nhất, được thiết kế để vận hành trên mặt đất hoặc trong không gian trong nhà như văn phòng, nhà kho. Được sử dụng nhiều trong các lĩnh vực như logistics và sản xuất, dịch vụ giao hàng, vệ sinh tự động.

- Robot dưới nước: hay còn gọi là phương tiện dưới nước tự hành – AUVs (Autonomous Underwater Vehicles). Những robot này hoạt động trong môi trường nước để thực hiện các nhiệm vụ khảo sát, kiểm tra hoặc cứu hộ dưới đáy biển. Ví dụ: tàu ngầm không người lái, robot khảo sát đại dương.

b. Phân loại dựa trên thiết bị di chuyển

Robot di động có thể được phân loại trên cơ sở các thiết bị họ sử dụng để hoạt động như sau:

- Robot có bánh xe: đây là loại robot phổ biến nhất, sử dụng bánh xe để di chuyển trên các bề mặt. Có nhiều loại robot có bánh xe khác nhau như robot một bánh, robot hai bánh.
- Robot có chân: loại robot sử dụng các chi (chân) khớp nối để di chuyển, thường được thiết kế để hoạt động trong các địa hình phức tạp mà bánh xe không thể tiếp cận. Ví dụ: robot 4 chân, robot 6 chân.
- Robot hình người: là những robot có hình dạng và cấu trúc gần giống con người. Chúng được trang bị nhiều cảm biến để mô phỏng hành vi người thật. Ví dụ: robot phục vụ, robot trợ lý trong chăm sóc người cao tuổi.

1.1.3 Ứng dụng

Robot di động hiện đang được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống và sản xuất nhờ vào khả năng di chuyển linh hoạt, tự chủ, cũng như tính thích nghi cao với môi trường. Các ứng dụng tiêu biểu bao gồm:

- Logistics và kho vận: robot di động được sử dụng để vận chuyển hàng hóa trong các trung tâm phân phối, kho hàng thông minh hoặc dây chuyền sản xuất, góp phần tự động hóa quy trình và tăng hiệu suất vận hành.



Hình 1.1. Robot được sử dụng trong vận chuyển hàng hóa trong kho.

- Dịch vụ và chăm sóc y tế: robot phục vụ trong nhà hàng, khách sạn, hoặc robot chăm sóc tại bệnh viện, viện dưỡng lão,... giúp giảm tải công việc cho con người và tăng trải nghiệm người dùng.



Hình 1.2. Robot giao hàng tự động được triển khai tại bệnh viện ở Singapore.

- Vệ sinh và lau dọn tự động: các robot hút bụi, robot lau sàn là ví dụ điển hình trong lĩnh vực nhà ở thông minh. Chúng hoạt động hiệu quả trong môi trường trong nhà, giúp tối ưu hóa thời gian dọn dẹp.



Hình 1.3. Robot hút bụi, lau sàn.

- Khảo sát và giám sát: robot di động được sử dụng trong khảo sát địa hình, giám sát môi trường, kiểm tra cơ sở hạ tầng hoặc làm nhiệm vụ trong các khu vực nguy hiểm như khu vực có bức xạ, đổ nát, hỏa hoạn.



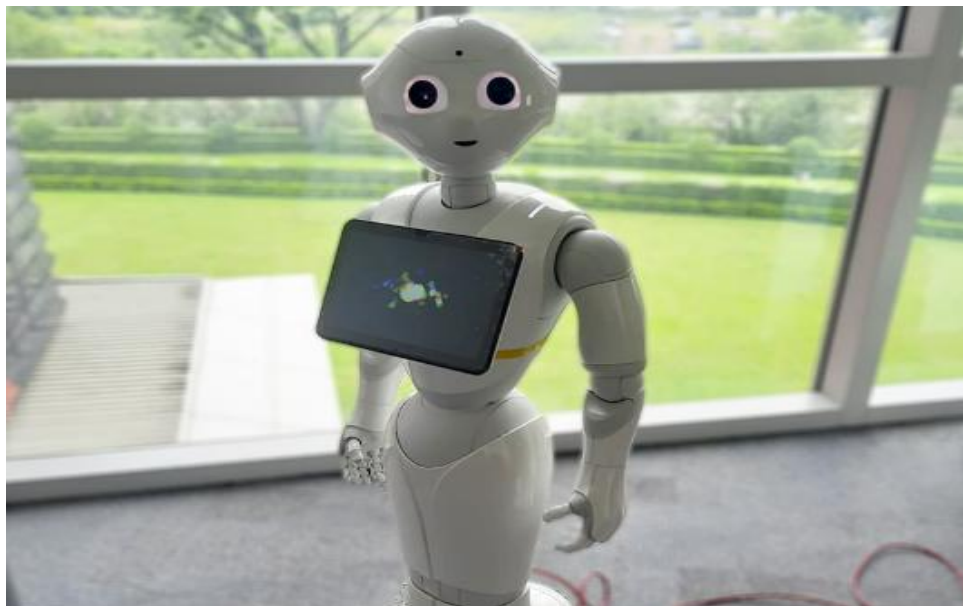
Hình 1.4. Robot Spirit được thiết kế để khám phá địa hình giống như trên mặt trăng.

- Quân sự và cứu hộ: trong lĩnh vực quốc phòng, robot di động có thể được dùng để dò mìn, vận chuyển trang bị, hoặc hỗ trợ lực lượng cứu hộ tại hiện trường thiên tai, tai nạn.



Hình 1.5. Robot hình người thực hiện nhiệm vụ cứu hộ con người trong thảm họa.

- Giải trí và giáo dục: Một số robot hình người hoặc thú cưng giải trí được ứng dụng trong các chương trình giáo dục STEM, giúp học sinh tiếp cận với công nghệ lập trình, cơ điện tử, và trí tuệ nhân tạo.



Hình 1.6. Robot giáo dục SoftBank Pepper.

1.1.4 Thách thức

Robot di động đang ngày càng trở nên phổ biến với nhiều ứng dụng thực tiễn, tuy nhiên vẫn còn tồn tại những thách thức đáng kể như khả năng thích ứng với môi trường động, xử lý dữ liệu cảm biến trong thời gian thực và tối ưu hóa hiệu suất hoạt động tổng thể. Đặc biệt, trong bối cảnh hiện tại với sự đa dạng lớn của các thuật toán lập kế hoạch đường đi - từ thuật toán tìm kiếm đồ thị, thuật toán lấy mẫu ngẫu nhiên,

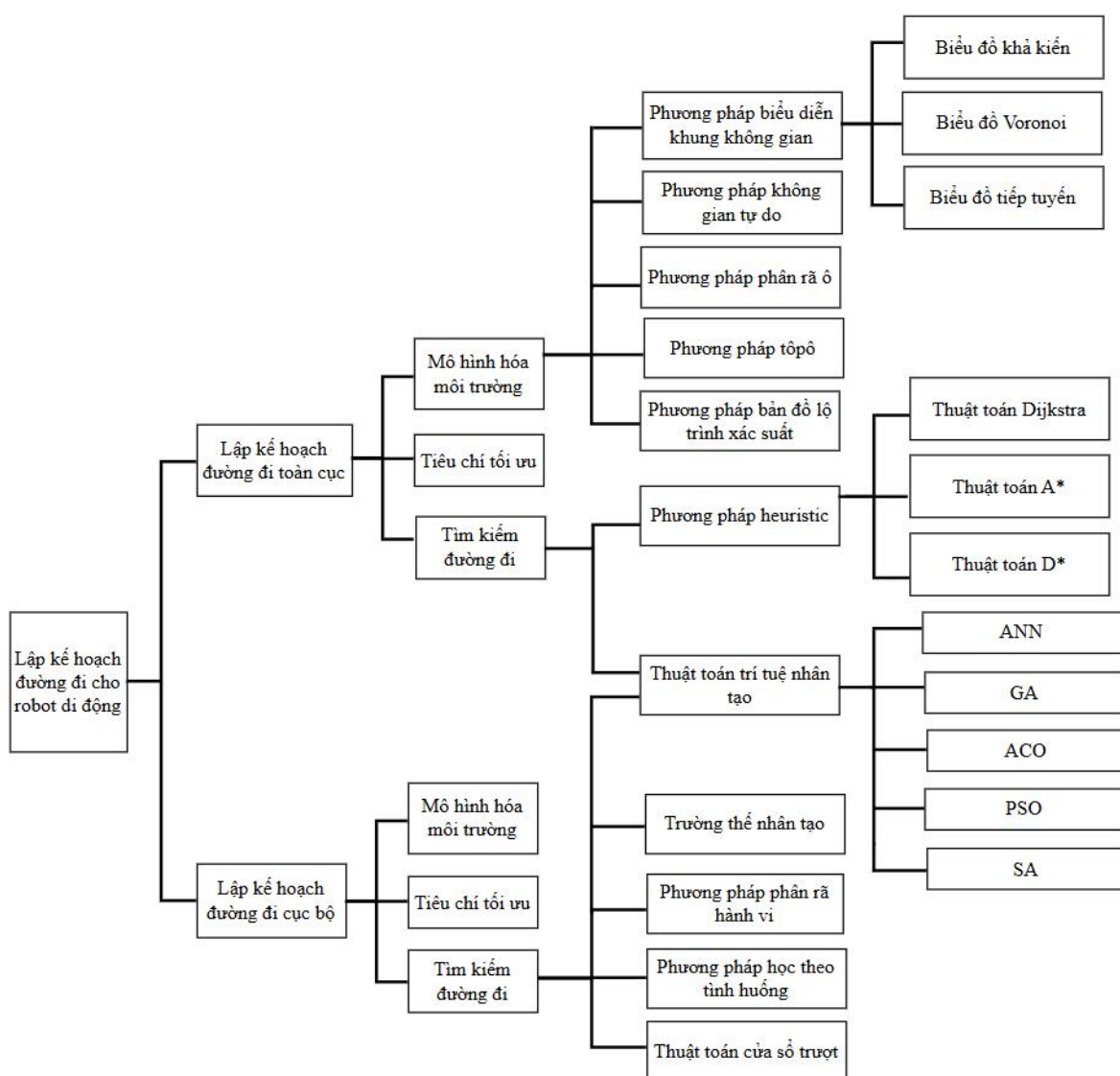
đến các phương pháp tối ưu hóa và học máy việc lựa chọn giải pháp phù hợp cho từng tình huống cụ thể không hề đơn giản. Lập kế hoạch đường đi là một trong những bài toán quan trọng nhất trong lĩnh vực robot di động, quyết định khả năng robot di chuyển từ vị trí hiện tại đến mục tiêu một cách an toàn, hiệu quả và tối ưu. Trong thực tế, robot thường phải hoạt động trong những môi trường có cấu trúc phức tạp, nhiều vật cản và giới hạn về thời gian hoặc năng lượng. Điều này đòi hỏi thuật toán tìm đường không chỉ tìm được lộ trình hợp lệ, mà còn phải tối ưu theo các tiêu chí cụ thể như quãng đường di chuyển, độ mượt mà, hay số bước xử lý. Đây cũng chính là điều mà đồ án này hướng đến.

1.2 Tổng quan về lập kế hoạch đường đi cho robot

1.2.1 Định nghĩa và phân loại

- Lập kế hoạch đường đi cho robot di động là một trong những chức năng cốt lõi trong hệ thống điều hướng của robot di động, nhằm tìm ra một quỹ đạo di chuyển tối ưu và an toàn từ vị trí xuất phát đến điểm đích. Quỹ đạo này cần đảm bảo tránh được chướng ngại vật, tiết kiệm thời gian di chuyển, giảm thiểu độ dài đường đi và chi phí năng lượng, đồng thời phù hợp với các ràng buộc môi trường và khả năng vận động của robot [3].
- Dựa trên mức độ nhận thức của robot về môi trường, bài toán lập kế hoạch đường đi thường được phân chia thành hai loại chính: lập kế hoạch đường đi toàn cục (global path planning) và lập kế hoạch đường đi cục bộ (local path planning) [4]:
 - Lập kế hoạch toàn cục là phương pháp trong đó robot đã biết trước toàn bộ bản đồ môi trường, bao gồm vị trí của các chướng ngại vật và mục tiêu. Dựa trên thông tin này, robot sử dụng các thuật toán tìm kiếm để tính toán một đường đi tối ưu hoặc gần tối ưu. Do được thực hiện trước khi robot bắt đầu di chuyển, phương pháp này còn được gọi là lập kế hoạch ngoại tuyến (offline) hoặc lập kế hoạch tĩnh (static path planning).
 - Lập kế hoạch cục bộ được áp dụng khi robot chưa biết hoặc chỉ biết một phần thông tin về môi trường. Trong quá trình di chuyển, robot sử dụng các cảm biến như Lidar hoặc camera để thu thập dữ liệu môi trường theo thời

gian thực, sau đó ra quyết định di chuyển dựa trên các thuật toán phản ứng. Do đặc điểm này, phương pháp còn được gọi là lập kế hoạch trực tuyến (online) hoặc lập kế hoạch động (dynamic path planning). Phương pháp này có tính linh hoạt cao, tuy nhiên có thể chỉ đạt được kết quả tối ưu cục bộ và trong một số trường hợp có thể dẫn đến thất bại trong việc tìm đến đích.



Hình 1.7. Phân loại lập kế hoạch đường đi

1.2.2 Các phương pháp mô hình hóa môi trường

Trước khi thực hiện lập kế hoạch đường đi toàn cục cho robot di động, cần xây dựng một mô hình môi trường phù hợp để:

- Hiểu rõ hơn các biến môi trường.

- Giảm bớt các kế hoạch không cần thiết.
- Giảm đáng kể khối lượng tính toán.

Các phương pháp phổ biến gồm:

- Phương pháp không gian khung (Framework Space Approach).
- Phương pháp không gian tự do (Free Space Approach).
- Phương pháp phân rã ô (Cell Decomposition Approach).
- Phương pháp tô pô (Topological Method).
- Phương pháp bản đồ đường đi xác suất (Probabilistic Roadmap Method).
- ***Phương pháp không gian khung (Framework Space Approach)***

Để đơn giản hóa bài toán, robot thường được mô hình hóa như một điểm, các vật cản được phóng to tỷ lệ sao cho robot có thể di chuyển tự do trong không gian chứa vật cản mà không xảy ra va chạm. Gồm 3 dạng:

- Đồ thị hiển thị (Visibility Graph).
- Đồ thị Voronoi (Voronoi Graph).
- Đồ thị tiếp tuyến (Tangent Graph).

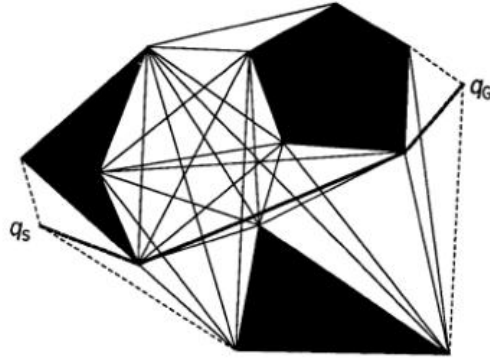
Đồ thị hiển thị (Visibility Graph)

Một đa giác được sử dụng để biểu diễn một vật cản trong phương pháp đồ thị hiển thị, và mỗi điểm cuối được nối với tất cả các đỉnh có thể nhìn thấy để tạo thành bản đồ cuối cùng. Trong phạm vi của đa giác, một đỉnh được nối với tất cả các điểm lân cận của nó, vì vậy robot di động có thể di chuyển dọc theo cạnh của đa giác. Tìm kiếm tập hợp các đoạn thẳng này và chọn một đường đi tối ưu từ điểm bắt đầu đến điểm kết thúc.

Phương pháp này có thể giải quyết hiệu quả bài toán kích thước nhỏ trong không gian hai chiều và đường đi là tối ưu, nhưng độ phức tạp thời gian là $O(N^2)$.

Tuy nhiên, khi độ phức tạp của bài toán tăng lên, hiệu suất của đồ thị hiển thị sẽ giảm mạnh. Nếu đồ thị hiển thị được sử dụng trong không gian ba chiều hoặc

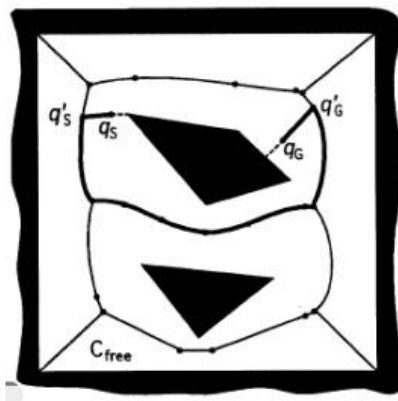
không gian có bậc cao hơn, thì đây là một bài toán NP-khó. Đồng thời, vì robot di động có kích thước và hình dạng nhất định, nên tất cả các đường đi đi qua mép của vật cản, do đó đường đi thu được rất có thể sẽ va chạm [4].



Hình 1.8. Đồ thị hiển thị (Visibility Graph)

Đồ thị Voronoi (Voronoi Graph)

Đồ thị Voronoi là quỹ đạo của các điểm cách đều từ hai hoặc nhiều ranh giới vật cản gần nhất, bao gồm cả ranh giới không gian làm việc. Tập hợp các đỉnh được tạo thành từ các điểm cách đều từ ba hoặc nhiều ranh giới vật cản, trong khi tập hợp các cạnh được tạo thành từ các điểm cách đều từ đúng hai ranh giới vật cản. Ưu điểm của đồ thị Voronoi là tốc độ tính toán nhanh, còn nhược điểm là có nhiều vị trí biến thiên [4].

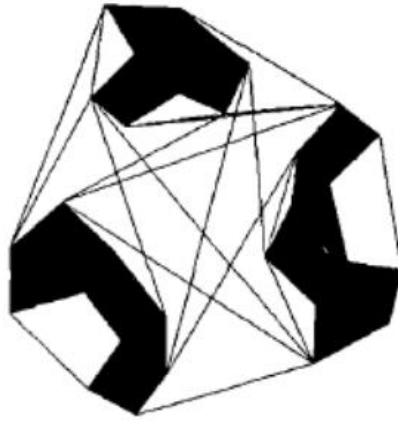


Hình 1.9. Đồ thị Voronoi (Voronoi Graph)

Đồ thị tiếp tuyến (Tangent Graph)

Trong đồ thị tiếp tuyến, các nút biểu diễn các điểm tiếp tuyến trên ranh giới của vật cản, và các cạnh biểu diễn các tiếp tuyến chung không xung đột giữa các vật

cản hoặc giữa các đoạn biên lồi của vật cản. Đồ thị tiếp tuyến yêu cầu bộ nhớ $O(K^2)$, trong đó K là số đoạn lồi của ranh giới vật cản. Nhược điểm là nếu xảy ra lỗi vị trí trong quá trình điều khiển, thì khả năng robot va chạm với vật cản sẽ rất cao [4].



Hình 1.10. Đồ thị tiếp tuyến (Tangent Graph)

- ***Phương pháp không gian tự do (Free Space Approach)***

Dựa trên khái niệm liên kết tự do, Habib và cộng sự đã phát triển một kỹ thuật mới để xây dựng không gian tự do có thể sử dụng giữa các vật cản trong môi trường robot dưới dạng vùng lồi tự do. Sau đó, một đồ thị mới có tên MAKLINK được xây dựng để tạo ra một đường đi không xung đột. Đồ thị được xây dựng bằng cách sử dụng trung điểm của các liên kết tự do giữa các vùng lồi tự do làm điểm trung gian. Các điểm này đại diện cho các nút, và các kết nối giữa các điểm trong cùng một vùng lồi đại diện cho các cung trong đồ thị. Một đường đi không xung đột có thể được tạo ra hiệu quả bằng cách sử dụng đồ thị MAKLINK. Độ phức tạp của việc tìm kiếm một đường đi không xung đột được giảm đáng kể bằng cách giảm kích thước đồ thị cần tìm kiếm - về số lượng nút và cung nối giữa chúng. Ưu điểm của phương pháp không gian tự do là tính linh hoạt cao, dễ duy trì sơ đồ mạng. Ngoài ra, nó có thể linh hoạt thay đổi điểm bắt đầu và điểm đích của robot. Tuy nhiên, trong môi trường có mật độ vật cản cao, phương pháp này có thể thất bại và không thể tìm được đường đi tối ưu [4].

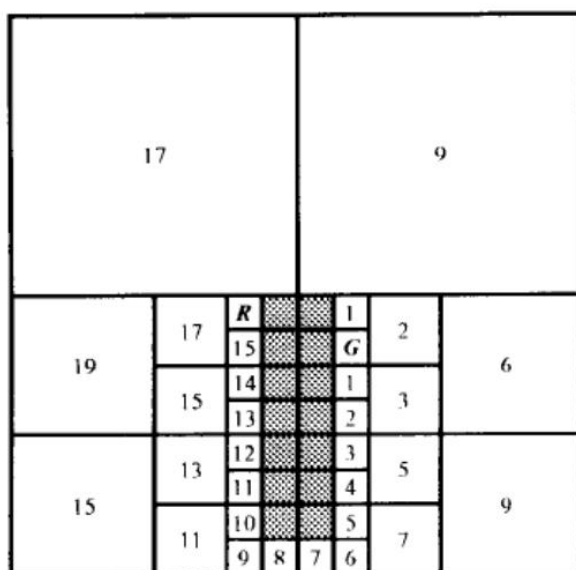
- ***Phương pháp phân rã ô (Cell Decomposition Approach)***

Phương pháp này phân rã không gian làm việc của robot thành một số vùng đơn

giản, và mỗi vùng được gọi là một ô. Các ô này tạo thành một đồ thị liên kết và một đường đi được tìm từ ô ban đầu đến ô mục tiêu. Nói chung, đường đi được biểu diễn bằng chỉ số thứ tự của ô. Phương pháp này được chia thành hai loại: phân rã ô chính xác và phân rã ô xấp xỉ.

- Với phân rã ô chính xác, không gian tự do được chia thành n đơn vị không chồng lấp. Sau khi kết hợp n đơn vị này, không gian thu được bằng với không gian tự do ban đầu.
- Trong phân rã ô xấp xỉ, tất cả các ô đều có dạng hình học định trước. Toàn bộ môi trường được chia thành một số hình chữ nhật lớn, mỗi hình chữ nhật là liên tục. Nếu bất kỳ hình chữ nhật lớn nào chứa vật cản hoặc biên, nó được chia nhỏ thành 4 hình chữ nhật nhỏ, và các hình nhỏ hơn sẽ được xử lý.

Quá trình này được lặp lại cho đến khi đạt đến biên của giải pháp. Cấu trúc này được gọi là cây tứ phân (quadtree) [4].



Hình 1.11. Phương pháp tiếp cận dựa trên cây tứ phân

- **Phương pháp tô pô (Topological Method)**

Phương pháp tô pô là một phương pháp giảm số chiều, và bài toán lập kế hoạch đường đi trong không gian hình học nhiều chiều được chuyển thành bài toán phân biệt về tính liên thông trong không gian thấp chiều. Khi mạng tô pô được thiết lập, đường đi được xác định từ điểm bắt đầu đến điểm đích. So với phương pháp phân

rã ô, phương pháp này chỉ cần ít thời gian xây dựng mô hình và ít không gian lưu trữ hơn. Độ phức tạp của phương pháp tôpô chỉ phụ thuộc vào số lượng vật cản, nên có thể đạt được lập kế hoạch nhanh. Phương pháp này phù hợp với môi trường có đặc điểm rõ ràng và vật cản thưa thớt; ngược lại, rất khó để điều hướng chính xác. Một nhược điểm khác là: phương pháp thu nhận thông tin môi trường trong phương pháp tôpô khó duy trì, khi số lượng vật cản tăng hoặc giảm thì mạng rất khó sửa đổi, vì quá trình thiết lập mạng tôpô tự nó đã rất phức tạp [4].

- ***Phương pháp bản đồ đường đi xác suất (Probabilistic Roadmap Method)***

Ý tưởng chính của phương pháp này như sau: Dựa trên lấy mẫu ngẫu nhiên, một đồ thị không có hướng $R = (N, E)$ được xây dựng, trong đó N là tập các nút được tạo ra từ quá trình lấy mẫu, E là tập các cạnh nối giữa các nút. Cho điểm bắt đầu s và điểm kết thúc f , phương pháp này sẽ tìm hai nút s' và f' sao cho s nối với s' , f' nối với f . Đường đi được lập bằng cách tìm chuỗi các cạnh nối từ s' đến f' trong đồ thị không có hướng [4].

1.3 Thuật toán A*

1.3.1 Tổng quan thuật toán A*

- ***Cơ sở hình thành***

Thuật toán A* được giới thiệu lần đầu tiên vào năm 1968 bởi ba nhà khoa học Peter Hart, Nils Nilsson và Bertram Raphael trong công trình nghiên cứu về các phương pháp tìm kiếm trên đồ thị tại Viện nghiên cứu Stanford (nay là SRI International). Trong bài báo gốc, thuật toán được gọi là thuật toán A (Algorithm A). Sau đó, với việc bổ sung một hàm heuristic phù hợp giúp đạt được kết quả tối ưu trong quá trình tìm kiếm, phiên bản cải tiến được đặt tên là A* để phản ánh hiệu suất vượt trội và tính tối ưu hóa của thuật toán [5].

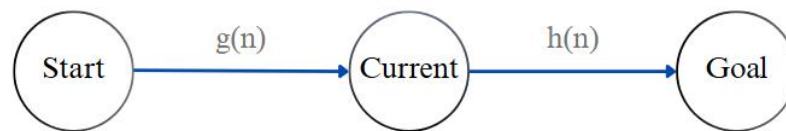
A* là một trong những thuật toán tìm đường hiệu quả và phổ biến nhất trong lĩnh vực trí tuệ nhân tạo, đặc biệt trong các bài toán lập kế hoạch đường đi cho robot di động. Thuật toán A* là sự kết hợp một cách hiệu quả giữa chi phí thực tế từ điểm bắt đầu đến điểm hiện tại và dự đoán chi phí còn lại từ điểm hiện tại đến đích có sử dụng hàm để ước lượng khoảng cách hay còn gọi là hàm Heuristic. Chính sự kết

hợp này giúp A* tìm ra lộ trình tối ưu với tốc độ nhanh chóng và độ chính xác cao.

- **Cấu trúc**

Thuật toán A* được xây dựng dựa trên 3 thành phần quan trọng:

$$f(n) = g(n) + h(n)$$



Hình 1.12. Minh họa hàm đánh giá $f(n)$ trong thuật toán A*

- $g(n)$: chi phí thực tế từ điểm bắt đầu đến điểm hiện tại. Đây là tổng chi phí đã tích lũy trên quãng đường đã đi qua.
- $h(n)$: ước lượng chi phí từ điểm hiện tại đến đích. Đây là thành phần heuristic giúp hướng dẫn quá trình tìm kiếm.
- $f(n)$: tổng chi phí ước lượng để đi từ điểm bắt đầu đến điểm đích qua điểm hiện tại. Hàm $f(n)$ đóng vai trò quyết định trong việc lựa chọn điểm tiếp theo để di chuyển, ưu tiên mở rộng các điểm có giá trị $f(n)$ thấp nhất, nhằm đảm bảo khả năng tìm thấy đường đi ngắn và nhanh nhất đến đích.

1.3.2 Nguyên lý hoạt động

Thuật toán A* hoạt động bằng cách mở rộng các nút từ điểm bắt đầu đến điểm kết thúc cho đến khi tìm được đường đi tốt nhất dựa trên hàm đánh giá $f(n)$. Quy trình được chia thành các bước như sau:

- **Khởi tạo**

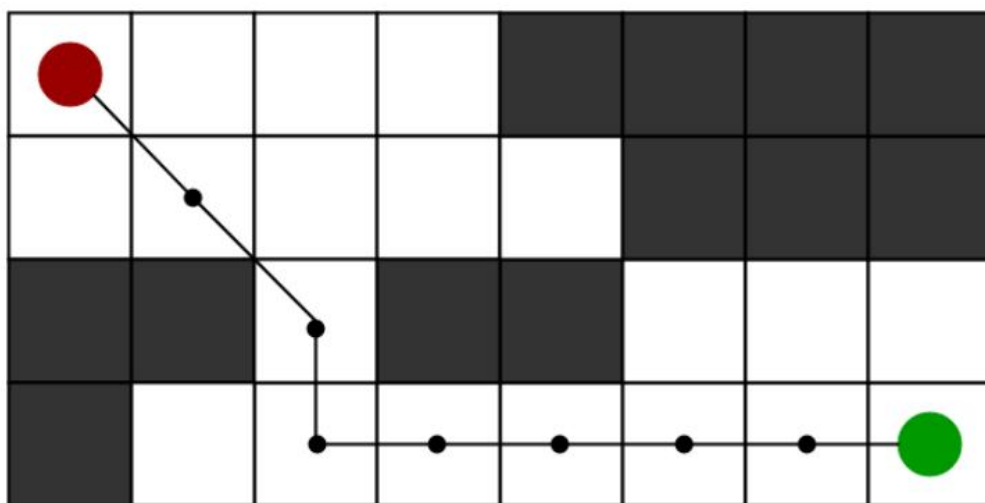
- Khởi tạo node bắt đầu, đưa vào danh sách mở (open list) và gán giá trị $f(n)$ ban đầu cho nó.
- Đặt danh sách đóng (closed list) rỗng.

- **Lặp lại**

- Chọn node trong danh sách mở có giá trị $f(n)$ nhỏ nhất:

- Nếu node này là đích $\rightarrow$ dừng thuật toán, truy vết đường đi.
- Nếu không:
 - Duyệt các node lân cận chưa duyệt
 - Tính toán $g(n)$, $h(n)$, $f(n)$ cho node kề cận. Sau đó, giá trị $f(n)$ này sẽ được so sánh với giá trị $f(n)$ của điểm đã được kiểm tra trước đó.
 - Nếu node này chưa có trong danh sách mở hoặc có giá trị $f(n)$ nhỏ hơn giá trị hiện tại trong danh sách mở, thì cập nhật giá trị và đặt node này vào danh sách mở.
- **Kết thúc**
 - Quá trình tìm kiếm sẽ tiếp tục cho đến khi xảy ra một trong 2 điều kiện sau:
 - Thuật toán tìm thấy điểm đích và đường đi tối ưu được xác định.
 - Danh sách mở rỗng, đường đi không tồn tại.

Dưới đây minh họa một ví dụ đơn giản về đường đi được tạo ra bởi thuật toán A*. Thuật toán bắt đầu từ ô xuất phát (chấm đỏ) và tiến hành mở rộng các ô kề cận không phải vật cản (ô trắng) nhằm tìm ra lộ trình tối ưu đến ô đích (chấm xanh). Các bước đi được xác định dựa trên hàm đánh giá chi phí tổng hợp, đồng thời đảm bảo tránh va chạm với vật cản (ô đen).



Hình 1.13. Minh họa đường đi được tạo bởi thuật toán A* trên bản đồ dạng lưới

1.3.3 Ưu và nhược điểm

- *Ưu điểm:*
 - Tối ưu: A* đảm bảo tìm được đường đi ngắn nhất nếu hàm heuristic $h(n)$ không vượt quá chi phí thực tế từ điểm hiện tại đến điểm đích.
 - Hiệu quả: A* thường rất nhanh và tìm ra đường đi ngắn nhất nhờ sử dụng thông tin heuristic.
 - Có thể áp dụng cho cả môi trường 2D/3D, có thể tùy biến theo ràng buộc môi trường.
- *Nhược điểm:*
 - Tốn bộ nhớ: cần lưu nhiều node trong danh sách mở và đóng.
 - Phụ thuộc vào heuristic: nếu chọn không phù hợp, hiệu suất giảm rõ rệt.
 - Đường đi chưa trơn mượt: có thể sinh nhiều khúc cua không cần thiết, ảnh hưởng đến hiệu quả vận hành robot.

1.3.4 Ứng dụng trong lập kế hoạch đường đi

Thuật toán A* là một trong những công cụ mạnh mẽ và hiệu quả để tìm đường ngắn nhất, được ứng dụng rộng rãi trong nhiều lĩnh vực của cuộc sống và công nghệ. Ứng dụng thực tiễn phổ biến:

- Định Tuyến GPS: A* được ứng dụng để xác định lộ trình ngắn nhất và tối ưu giữa hai điểm trên bản đồ, giúp định tuyến hiệu quả trong các hệ thống dẫn đường.
- Lập Trình Game: trong lĩnh vực phát triển game, thuật toán A* thường được sử dụng để giúp các nhân vật tự động tìm đường di chuyển thông minh, tránh vật cản và tương tác mượt mà với môi trường.
- Robot di động: A* đóng vai trò quan trọng trong việc xác định đường đi tối ưu từ vị trí hiện tại đến đích, đảm bảo an toàn và hiệu quả vận hành.

1.4 Thuật toán tham lam

1.4.1 Tổng quan thuật toán

- *Khái niệm*

Tham lam là một mô hình thuật toán xây dựng giải pháp từng phần, luôn chọn phần

tiếp theo mang lại lợi ích rõ ràng và tức thời nhất. Phương pháp này thường được sử dụng trong các bài toán tối ưu hóa, với mục tiêu tối đa hóa lợi ích hoặc tối thiểu hóa chi phí [6].

- *Tính chất*

Một bài toán tối ưu có thể được giải bằng thuật toán tham lam nếu nó thỏa mãn hai tính chất quan trọng sau:

- Tính chất lựa chọn tham lam: có thể xây dựng lời giải tối ưu bằng cách liên tiếp đưa ra các lựa chọn tối ưu cục bộ tại mỗi bước.
- Cấu trúc tối ưu con: lời giải tối ưu cho bài toán ban đầu bao gồm các lời giải tối ưu của các bài toán con phát sinh trong quá trình giải quyết. Do đó, giải quyết các vấn đề cục bộ sẽ góp phần vào giải pháp tổng thể.

- *Đặc điểm*

- Thuật toán tham lam đơn giản và dễ triển khai.
- Luôn chọn phương án có lợi nhất ở thời điểm hiện tại mà không xét đến toàn bộ bài toán.
- Thuật toán nhanh và hiệu quả với độ phức tạp về thời gian là $O(n \log n)$ hoặc $O(n)$. Do đó được ứng dụng vào việc giải các bài toán có quy mô lớn.
- Việc tìm kiếm giải pháp tối ưu được thực hiện không lặp lại – thuật toán chạy một lần.

1.4.2 Nguyên lý hoạt động

Thuật toán tham lam giải quyết các vấn đề tối ưu hóa bằng cách đưa ra lựa chọn cục bộ tốt nhất tại mỗi bước với hy vọng tìm ra tối ưu toàn cục. Giống như việc chọn phương án tốt nhất có sẵn tại mỗi thời điểm, hy vọng nó sẽ dẫn đến kết quả tổng thể tốt nhất. Sau đây là quy trình hoạt động tổng quát:

- Bước 1: khởi tạo trạng thái ban đầu.
- Bước 2: từ trạng thái hiện tại, đánh giá tất cả các lựa chọn có thể thực hiện. Lựa chọn tùy chọn tốt nhất tại thời điểm hiện tại.

- Bước 3: di chuyển đến trạng thái mới dựa trên tùy chọn bạn đã chọn. Cập nhật trạng thái, loại bỏ bước đã chọn.
- Bước 4: lặp lại cho đến khi đạt đến điều kiện dừng: đạt đến trạng thái mục tiêu hoặc không còn lựa chọn nào.

1.4.3 Ưu và nhược điểm

- *Ưu điểm*
 - Tính đơn giản: thuật toán tham lam dễ hiểu và dễ triển khai vì chúng tuân theo cách tiếp cận trực tiếp là đưa ra lựa chọn tốt nhất ở mỗi bước.
 - Hiệu quả: chúng thường có độ phức tạp về thời gian thấp hơn so với các thuật toán khác như lập trình động, khiến chúng nhanh hơn đối với một số vấn đề nhất định.
 - Quyết định tối ưu cục bộ: bằng cách đưa ra những lựa chọn tối ưu cục bộ, các thuật toán tham lam có thể nhanh chóng tìm ra giải pháp mà không cần phải quay lại phức tạp.
 - Hoạt động tốt đối với một số vấn đề nhất định: thuật toán tham lam đặc biệt hiệu quả đối với các vấn đề thể hiện tính chất lựa chọn tham lam và cấu trúc phụ tối ưu, chẳng hạn như cây khung nhỏ nhất và mã hóa Huffman.
- *Nhược điểm*
 - Không phải lúc nào cũng tối ưu: thuật toán tham lam ưu tiên tối ưu cục bộ hơn tối ưu toàn cục, dẫn đến các giải pháp không tối ưu trong một số trường hợp.
 - Khó chứng minh tính tối ưu: việc chứng minh tính tối ưu của một thuật toán tham lam có thể rất khó khăn, đòi hỏi phải phân tích cẩn thận.
 - Nhạy cảm với thứ tự đầu vào: thứ tự dữ liệu đầu vào có thể ảnh hưởng đến giải pháp được tạo ra bởi thuật toán tham lam.
 - Khả năng áp dụng hạn chế: thuật toán tham lam không phù hợp với mọi vấn đề và có thể không áp dụng được cho các vấn đề có ràng buộc phức tạp.

1.4.4 Ứng dụng của thuật toán

Thuật toán tham lam được ứng dụng rộng rãi trong nhiều lĩnh vực của khoa học máy tính, đặc biệt là trong các bài toán tối ưu có cấu trúc phù hợp. Dưới đây là một số ứng dụng tiêu biểu:

- Bài toán cái túi phân số: chọn vật phẩm để tối đa hóa giá trị mang theo, nhưng tổng trọng lượng không vượt quá giới hạn.
- Thuật toán đường đi ngắn nhất Dijkstra: dùng để tìm đường đi ngắn nhất từ một đỉnh đến tất cả các đỉnh còn lại trong đồ thị trọng số không âm.
- Thuật toán cây bao trùm nhỏ nhất của Kruskal và Prim: tìm cây bao trùm nhỏ nhất cho một đồ thị có trọng số. Cây bao trùm nhỏ nhất được sử dụng trong thiết kế mạng máy tính và có nhiều ứng dụng trong thế giới thực.
- Mã hóa Huffman: dùng để mã hóa dữ liệu hiệu quả bằng cách gán mã ngắn cho ký tự xuất hiện nhiều.
- Bài toán xấp xỉ trong các vấn đề khó: trong bài toán Người bán hàng, thuật toán tham lam có thể dùng để tìm lời giải gần đúng bằng cách tại mỗi bước chọn thành phố chưa thăm gần nhất.

1.5 Thuật toán tối ưu hóa đàn kiến

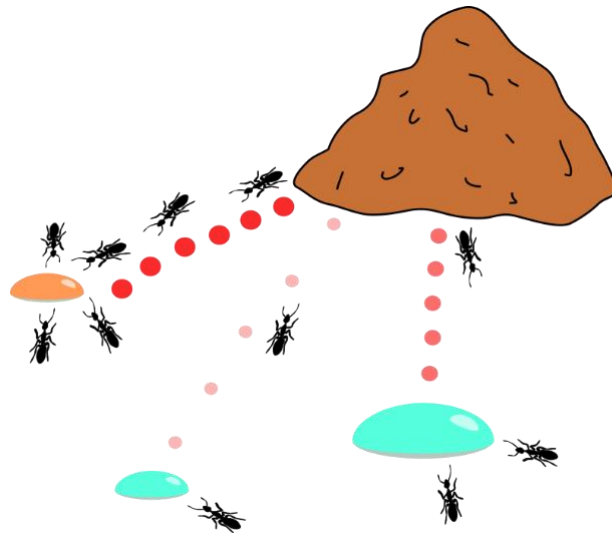
1.5.1 Tổng quan

- *Cơ sở hình thành*

Thuật toán tối ưu hóa đàn kiến (Ant Colony Optimization – ACO) là một thuật toán tối ưu thuộc nhóm metaheuristic mô phỏng quá trình di chuyển của đàn kiến trong tự nhiên, được giới thiệu lần đầu tiên vào năm 1992 bởi Marco Dorigo. Thuật toán này lấy cảm hứng từ hành vi kiếm ăn tập thể của kiến trong tự nhiên. nơi mà các cá thể kiến có khả năng tìm được đường đi ngắn nhất từ tổ đến nguồn thức ăn nhờ vào cơ chế để lại và lần theo pheromone (vết mùi).

Khi một con kiến di chuyển, nó sẽ để lại pheromone trên đường đi. Các kiến đi sau có xu hướng ưu tiên chọn những đường có nồng độ pheromone cao hơn. Qua thời gian, nếu nhiều con kiến tiếp tục sử dụng cùng một lộ trình, nồng độ pheromone trên đường đó sẽ được tăng cường, trong khi những đường ít được đi qua sẽ dần

mất pheromone do bay hơi. Quá trình này dẫn đến việc cả đàn kiến hội tụ vào những đường đi ngắn và hiệu quả nhất.



Hình 1.14. Mô phỏng hành vi tìm kiếm thức ăn của bầy kiến trong tự nhiên.

Trong mô hình tính toán, mỗi con kiến nhân tạo đại diện cho một lời giải tiềm năng trong không gian tìm kiếm (thường là một đồ thị). Dựa trên mức pheromone tích lũy trên các cạnh của đồ thị, các kiến sẽ ưu tiên đi theo các con đường có xác suất cao hơn, được xác định bởi nồng độ pheromone và độ hấp dẫn (heuristic) của đường đi.

Mặc dù từng con kiến chỉ có khả năng ra quyết định cục bộ, nhưng khi kết hợp hành vi của toàn đàn với cơ chế pheromone, bầy kiến có thể khám phá và tối ưu hóa con đường một cách hiệu quả. Ý tưởng này được đưa vào mô hình tính toán nhằm tối ưu các bài toán tổ hợp phức tạp, điển hình như Traveling Salesman Problem (TSP) [7].

- *Cấu trúc*

Để mô phỏng hành vi của các con kiến thực, Dorigo xây dựng các con kiến nhân tạo cũng có đặc trưng sản sinh ra vết mùi để lại trên đường đi và khả năng lần vết theo nồng độ mùi để lựa chọn con đường có nồng độ mùi cao hơn để đi. Gắn với mỗi cạnh (i, j) được gán hai thông số: nồng độ pheromone trên cạnh τ_{ij} và thông tin heuristic η_{ij} thường là nghịch đảo khoảng cách giữa hai đỉnh.

Đặc điểm chính của thuật toán này là tại mỗi vòng lặp, tất cả m con kiến đều tham

gia vào việc cập nhật lượng pheromone trên đồ thị. Lượng pheromone τ_{ij} liên kết với cạnh nối giữa hai thành phố i và j , được cập nhật theo công thức sau:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \sum_{k=1}^m \Delta \tau_{ij}^k \quad (1-1)$$

Trong đó:

- ρ : tỷ lệ bay hơi của vết mùi.
- m : số lượng kiến.
- $\Delta \tau_{ij}^k$: lượng pheromone là con kiến k để lại trên cạnh (i, j) , được xác định như sau:

$$\Delta \tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{Nếu con kiến } k \text{ đi qua cạnh } (i, j), \\ 0 & \text{Ngược lại,} \end{cases} \quad (1-2)$$

- Q : một hằng số.
- L_k : là độ dài hành trình mà kiến k đã đi qua trong vòng lặp hiện tại.

Trong quá trình xây dựng lời giải, phương tiện truyền đạt tín hiệu được kiến sử dụng để thông báo cho những con khác trong việc tìm đường đi hiệu quả nhất chính là mùi của chúng (pheromone). Các con kiến lựa chọn đỉnh kế tiếp để di chuyển tới thông qua một cơ chế ngẫu nhiên có xác suất. Khi kiến k đang ở đỉnh i thì xác suất để kiến:

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{j \in N_i^k} [\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}, j \in N_i^k \quad (1-3)$$

0

Trong đó:

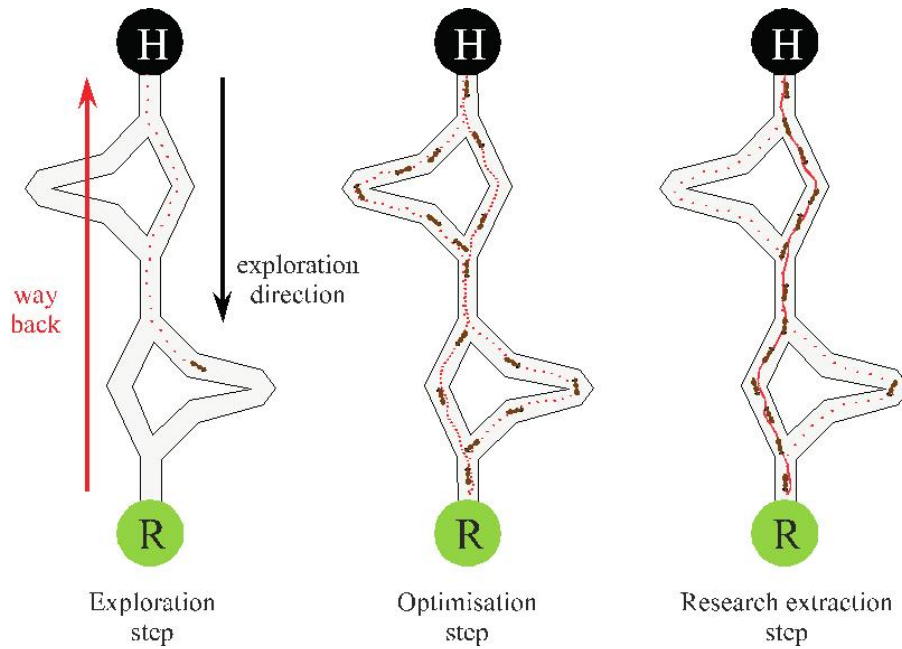
- p_{ij}^k : xác suất con kiến k lựa chọn cạnh i, j .
- α : hệ số điều chỉnh ảnh hưởng của pheromone τ_{ij} .
- β : hệ số điều chỉnh ảnh hưởng của thông tin heuristic η_{ij} .
- N_i^k : tập hợp các đỉnh láng giềng mà kiến k có thể đi được từ đỉnh i .

- η_{ij} : thông tin heuristic giúp đánh giá chính xác sự lựa chọn của con kiến khi quyết định đi từ đỉnh i qua đỉnh j , được các đỉnh theo công thức:

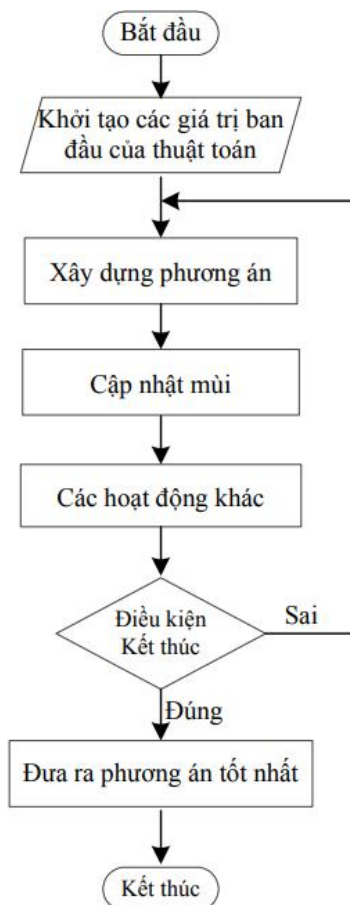
$$\eta_{ij} = \frac{1}{d_{ij}} \quad (1-4)$$

- d_{ij} : khoảng cách giữa đỉnh i và đỉnh j .

1.5.2 Nguyên lý hoạt động



Hình 1.15. Cơ chế lựa chọn và củng cố đường đi tối ưu trong thuật toán ACO.



Hình 1.16. Lưu đồ hoạt động của thuật toán ACO

Thuật toán tối ACO là một thuật toán metaheuristic mô phỏng hành vi kiếm ăn của đàn kiến nhằm giải các bài toán tối ưu. Dưới đây là các bước chính minh họa cách ACO hoạt động:

- Khởi tạo: ACO bắt đầu bằng việc thiết lập cấu trúc đồ thị tương ứng với bài toán cụ thể và khởi tạo các vết mùi (pheromone) trên các cạnh.
- Xây dựng lời giải: các kiến nhân tạo bắt đầu di chuyển qua đồ thị từ một điểm xuất phát được chọn, theo cách xác suất, dựa trên nồng độ pheromone hiện tại và thông tin heuristic. Chúng xây dựng lời giải bằng cách lặp lại việc chọn cạnh và ghé thăm các đỉnh.
- Cập nhật pheromone: sau khi mỗi con kiến hoàn thành hành trình của mình, các vết pheromone trên các cạnh được cập nhật. Lượng pheromone được tăng cường tùy thuộc vào chất lượng lời giải mà kiến đó tìm được.
- Cân bằng thăm dò – khai thác: ACO cân bằng giữa việc tìm đường mới và tăng

cường đường tốt bằng cách kết hợp cả pheromone và thông tin heuristic.

- **Lặp lại và dừng:** quá trình xây dựng lời giải và cập nhật pheromone được lặp lại nhiều lần. Thuật toán kết thúc khi đạt đến một điều kiện dừng:
 - Đã lặp đến số vòng tối đa.
 - Tìm được lời giải thỏa mãn.

Thông qua quá trình lặp lại, thuật toán ACO dần hội tụ về các lời giải chất lượng cao, nhờ vào sự kết hợp hiệu quả giữa việc khai thác kinh nghiệm tích lũy từ pheromone và khả năng khám phá không gian nghiệm mới.

1.5.3 Ưu và nhược điểm

- **Ưu điểm**
 - Khả năng tìm nghiệm gần tối ưu: ACO có hiệu quả cao trong việc tìm ra các nghiệm gần tối ưu, ngay cả trong không gian tìm kiếm phức tạp. Bằng cách khai thác trí tuệ tập thể của các kiến nhân tạo và cơ chế tăng cường pheromone, thuật toán có thể khám phá và khai thác hiệu quả các vùng tiềm năng trong không gian lời giải.
 - Tính mở rộng cao: với bản chất tính song song của việc tìm kiếm từ nhiều cá thể kiến, ACO dễ dàng được mở rộng để áp dụng cho các bài toán có quy mô lớn, nhiều biến và ràng buộc phức tạp.
 - Tính ổn định và bền vững: ACO thể hiện độ ổn định tốt ngay cả khi phải làm việc với dữ liệu không hoàn chỉnh hoặc nhiễu. Cơ chế pheromone kết hợp với thông tin heuristic giúp hướng dẫn việc ra quyết định, giảm ảnh hưởng từ sự thiếu chính xác trong mô hình.
 - Cân bằng giữa khám phá và khai thác: thuật toán duy trì sự cân bằng giữa việc khám phá các hướng đi mới và khai thác những hướng đã chứng minh hiệu quả. Điều này giúp ACO thoát khỏi các cực trị cục bộ và phát hiện các lời giải đa dạng, mở rộng khả năng tìm kiếm.
- **Nhược điểm**
 - Nguy cơ hội tụ sớm vào nghiệm kém: trong không gian lời giải có nhiều

cực trị cục bộ, thuật toán có thể bị hội tụ sớm vào những lời giải không tối ưu. Điều này là do cơ chế tăng cường pheromone có thể khóa hướng đi vào một vài tuyến đường được đánh giá cao sớm, làm giảm tính đa dạng trong tìm kiếm.

- Nhạy cảm với tham số: hiệu quả của thuật toán ACO phụ thuộc nhiều vào việc lựa chọn và hiệu chỉnh các tham số, đặc biệt là: tốc độ bay hơi pheromone, hệ số ảnh hưởng của pheromone và heuristic, số lượng kiến. Việc lựa chọn không phù hợp có thể làm giảm chất lượng lời giải hoặc làm thuật toán hội tụ chậm.
- Độ phức tạp tính toán cao: với các bài toán có kích thước lớn, việc quản lý số lượng lớn cá thể kiến và cập nhật liên tục pheromone có thể dẫn đến chi phí tính toán lớn về thời gian và bộ nhớ. Điều này khiến ACO không phù hợp với các ứng dụng yêu cầu thời gian thực hoặc tài nguyên hạn chế.

1.5.4 Ứng dụng của thuật toán

Thuật toán ACO đã chứng minh tính hiệu quả vượt trội trong việc giải quyết các bài toán tối ưu phức tạp trong thực tiễn. Nhờ khả năng tìm kiếm gần tối ưu, thích nghi tốt và khả năng mở rộng, ACO được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau. Một số ứng dụng tiêu biểu có thể kể đến như sau:

- Tối ưu hóa lộ trình và logistics: ACO đã chứng minh hiệu quả cao trong việc giải bài toán người giao hàng TSP, nơi cần xác định hành trình ngắn nhất đi qua một tập hợp các điểm mà không lặp lại. Trong các hệ thống logistics và vận chuyển, thuật toán được dùng để tối ưu hóa tuyến giao hàng, quản lý kho, cũng như cải thiện hiệu suất vận hành chuỗi cung ứng. Đặc biệt, trong lĩnh vực robot, ACO có thể giúp xác định chuỗi đường đi tối ưu qua nhiều điểm trung gian trong môi trường có chướng ngại vật.
- Quản lý mạng và tối ưu dữ liệu: trong các hệ thống mạng máy tính hoặc mạng cảm biến không dây, thuật toán có thể tìm ra tuyến truyền dẫn dữ liệu ngắn nhất và hiệu quả nhất, giúp giảm độ trễ và phân phối tải đều trên mạng. Ngoài ra, ACO

còn được sử dụng để thiết kế cấu trúc mạng, định tuyến gói tin, và tối ưu hóa hiệu suất truyền thông trong các hệ thống thời gian thực hoặc mạng IoT.

- **Lập lịch và phân bổ tài nguyên:** ACO được sử dụng để giải quyết các bài toán lập lịch sản xuất, tối ưu hóa việc phân bổ tài nguyên trong hệ thống điện toán đám mây, hoặc lên lịch thi, xếp thời khóa biểu trong giáo dục. Khả năng xử lý bài toán phân tổ phức tạp một cách gần tối ưu và nhanh chóng khiến ACO trở thành lựa chọn phù hợp trong các môi trường có tính ràng buộc cao và biến động liên tục.

CHƯƠNG 2: BÀI TOÁN LẬP KẾ HOẠCH ĐƯỜNG ĐI CHO ROBOT DI ĐỘNG ĐA MỤC TIÊU, ĐA ĐÍCH ĐẾN

2.1 Phát biểu bài toán

Trong bối cảnh robot di động ngày càng được ứng dụng rộng rãi trong các hệ thống kho thông minh, bệnh viện, nhà máy và các nền tảng giao vận hiện đại, bài toán lập kế hoạch đường đi (path planning) đóng vai trò then chốt nhằm đảm bảo khả năng di chuyển chính xác, an toàn và hiệu quả. Đặc biệt, trong các môi trường phức tạp có nhiều vật cản và yêu cầu robot di chuyển qua nhiều vị trí mục tiêu, bài toán trở nên không chỉ đơn thuần là tìm đường ngắn nhất mà còn phải xét đến các yếu tố như: độ trơn tru của hành trình, số lần rẽ, hiệu quả tính toán và khả năng mở rộng khi số lượng điểm đích tăng lên.

Lập kế hoạch đường đi đa mục tiêu là một biến thể quan trọng trong lĩnh vực này, trong đó robot cần di chuyển qua một chuỗi các điểm mục tiêu trung gian trước khi đến đích cuối cùng. Bài toán đòi hỏi việc xác định một hành trình tối ưu không chỉ về tổng quãng đường mà còn giảm thiểu số lần rẽ và số lượng nút được mở rộng trong quá trình tìm kiếm, từ đó đảm bảo hành trình mượt mà, tiết kiệm năng lượng và có khả năng triển khai thực tế.

Trong môi trường dạng lưới, mỗi ô thể hiện một vùng không gian có thể di chuyển được hoặc chứa vật cản, bài toán lập kế hoạch đường đi cho robot di động cần giải quyết các mục tiêu sau:

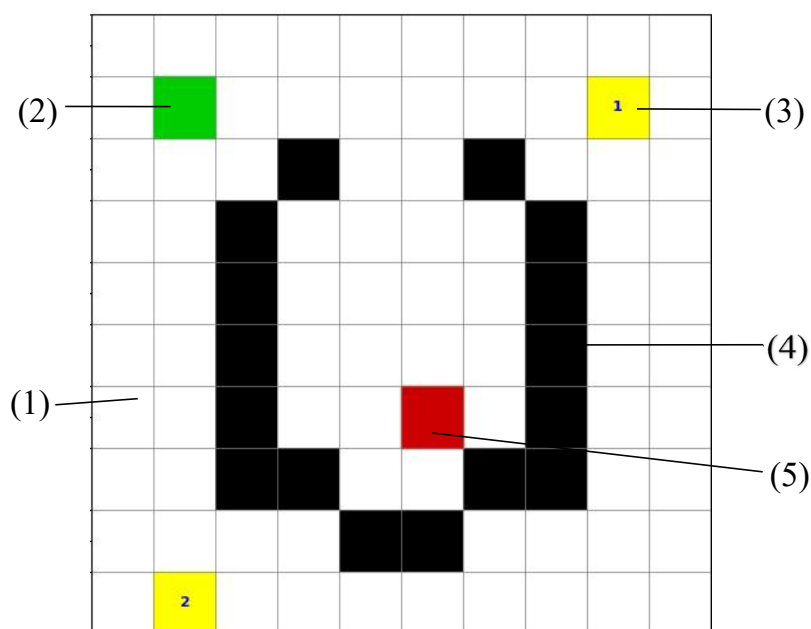
- Tổng độ dài quãng đường đi là nhỏ nhất.
- Số lần rẽ là tối thiểu để đảm bảo hành trình mượt mà.
- Số lượng nút được mở rộng trong quá trình tìm kiếm là ít nhất.
- Lộ trình đảm bảo tránh các vùng chứa vật cản.
- Robot có khả năng di chuyển qua nhiều điểm đích trung gian với thứ tự hợp lý.

Với những yêu cầu như vậy, các phương pháp truyền thống như A* có thể gặp hạn chế về hiệu năng và khả năng mở rộng. Do đó, việc nghiên cứu và đề xuất các chiến lược điều hướng hiệu quả hơn là cần thiết.

2.2 Mô hình hóa bài toán

Bài toán được đặt trong bối cảnh robot di động hoạt động trong môi trường 2D, có cấu trúc tĩnh – tức là không có sự thay đổi về vị trí chướng ngại vật trong suốt quá trình lập kế hoạch và di chuyển. Môi trường này phản ánh các không gian thực tế như kho hàng, xưởng sản xuất hoặc các khu vực vận hành trong nhà, nơi bố cục vật cản được xác định rõ ràng và không thay đổi trong quá trình robot thực hiện nhiệm vụ.

Để thuận tiện cho việc tính toán, môi trường được mô hình hóa dưới dạng bản đồ lưới (grid map) kích thước $M \times N$. Mỗi ô lưới được xác định duy nhất bởi cặp tọa độ (x, y) , qua đó mô tả chính xác vị trí không gian của từng điểm trong môi trường.



- (1) Ô màu trắng biểu diễn khu vực có thể đi
- (2) Ô màu xanh biểu diễn điểm xuất phát
- (3) Ô màu vàng biểu diễn các mục tiêu trung gian
- (4) Ô màu đen biểu diễn chướng ngại vật
- (5) Ô màu đỏ biểu diễn đích đến cuối cùng

Hình 2.1. Minh họa môi trường lưới 10×10 với nhiều đích đến và chướng ngại vật

Sử dụng ô lưới để chia môi trường ban đầu ra thành các ô. Sau đó, chuyển bản đồ thành một ma trận 2 chiều thể hiện tính chất của mỗi ô lưới như sau:

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- Đầu vào:
 - Một môi trường tĩnh dạng lưới 2D kích thước $M \times N$
 - Mỗi ô lưới đại diện cho một đơn vị không gian rời rạc.
 - Các ô có giá trị 0 biểu thị vùng có thể di chuyển.
 - Các ô có giá trị 1 biểu thị vùng bị chiếm dụng bởi vật cản.
 - Điểm xuất phát $S = (x_s, y_s)$: là vị trí bắt đầu của robot trên bản đồ.
 - Tập các điểm đích trung gian $P = \{X_1, X_2, \dots, X_n\}$: là các vị trí mà robot phải đi qua trong hành trình.
 - Điểm kết thúc $G = (x_t, y_t)$: là điểm cuối cùng mà robot cần đến.
- Đầu ra:
 - Đầu ra của bài toán là một lộ trình tối ưu cho robot, bắt đầu từ điểm khởi đầu (S), qua các điểm mục tiêu trung gian $(X_1, X_2, \dots, X_n)$, và kết thúc tại điểm đích (G).
- Mục tiêu:
 - Độ dài đường đi là ngắn nhất.
 - Đường đi mượt mà giảm số lần rẽ và tổng góc rẽ.
 - Số lượng nút được mở rộng tối thiểu trong quá trình tìm kiếm.

2.3 Các nghiên cứu liên quan

Các nghiên cứu trên đã đóng góp đáng kể vào việc giải quyết bài toán lập kế hoạch đường đi cho robot trong môi trường thực tế, đặc biệt trong các hệ thống có tính phức

tạp cao. Tuy nhiên, việc kết hợp hiệu quả giữa thuật toán tìm đường chính xác với các chiến lược chọn điểm mục tiêu thông minh vẫn còn là một hướng nghiên cứu tiềm năng.

Trong tài liệu [8], Bakdi áp dụng chiến lược theo từng bước để tìm đường đi tốt nhất. Đầu tiên, thuật toán di truyền được sử dụng để tạo ra đường đi không va chạm từ điểm xuất phát đến điểm đích. Sau đó, đa thức nội suy Hermite bậc ba được sử dụng để làm trơn đường đi.

Han sử dụng chiến lược tương tự trong tài liệu [9]. Ban đầu, thuật toán chọn điểm bao quanh được dùng để tìm đường đi không va chạm, sau đó thuật toán Particle Swarm Optimization (PSO) được áp dụng để cải thiện quỹ đạo. Kết quả cho thấy quỹ đạo được tạo ra có cải thiện rõ rệt về cả độ dài lẫn độ trơn mượt.

Davoodi áp dụng thuật toán NSGA-II để giải bài toán lập kế hoạch đường đi trong không gian rời rạc [10]. Trong đó, độ dài đường đi và độ an toàn được tối ưu đồng thời. Ahmed tối ưu cả ba tiêu chí – độ dài đường đi, an toàn và độ mượt – nhưng chỉ dùng các toán tử tiến hóa truyền thống. Do đó, trong các môi trường phức tạp, thuật toán cần nhiều cá thể hơn và nhiều vòng lặp hơn để tìm được lời giải tối ưu.

Ngoài ra, các phương pháp heuristic cũng đóng vai trò quan trọng trong việc tăng tốc quá trình tìm kiếm. Một số hướng tiếp cận nổi bật như: tăng cường hàm heuristic bằng các yếu tố hình học (góc rẽ, khoảng cách từ vật cản), điều chỉnh trọng số của hàm chi phí. Trong tài liệu [11], nghiên cứu này đề xuất một thuật toán A* cải tiến có tích hợp đặc điểm tham lam bằng cách điều chỉnh trọng số hàm heuristic và cài thêm chiến lược lựa chọn điểm mục tiêu linh hoạt nhằm tăng tốc quá trình tìm kiếm trong bài toán đa mục tiêu. Phương pháp này đã cho thấy hiệu quả tốt hơn A* truyền thống về mặt thời gian tính toán và độ mượt của đường đi, tuy nhiên vẫn còn gặp hạn chế về khả năng tối ưu toàn cục khi số lượng điểm mục tiêu lớn hoặc phân bố phức tạp.

Những nghiên cứu trước đây đã đặt nền móng quan trọng cho bài toán lập kế hoạch đường đi đa mục tiêu. Tuy nhiên, do các tiêu chí tối ưu thường mâu thuẫn nhau (ví dụ: đường ngắn nhưng nhiều rẽ gấp, hoặc đường an toàn nhưng dài), nên cần thiết phải có một phương pháp kết hợp thông minh vừa tận dụng khả năng tìm đường cục bộ hiệu quả như A*, vừa kế thừa sức mạnh toàn cục của các chiến lược metaheuristic như

ACO – đây cũng là định hướng chính của đề tài này.

CHƯƠNG 3. GIẢI THUẬT ĐỀ XUẤT

3.1 Thuật toán A* cải tiến kết hợp ACO (ImpA*ACO)

3.1.1 Lý do đề xuất kết hợp ImpA* với ACO

Trong bài toán lập kế hoạch đường đi đa mục tiêu, việc lựa chọn đúng thứ tự ghé thăm các điểm là yếu tố then chốt ảnh hưởng đến độ dài và hiệu quả của hành trình. Mặc dù A* cải tiến kết hợp thuật toán tham lam (ImpA*G) sử dụng chiến lược tham lam để đơn giản hóa vấn đề này, nó vẫn còn tồn tại những hạn chế.

- Nhược điểm của ImpA*G: chiến lược tham lam trong ImpA*G chỉ chọn điểm mục tiêu gần nhất tại mỗi bước, dẫn đến khả năng rơi vào cực trị cục bộ, tạo ra hành trình không tối ưu toàn cục, đặc biệt trong các bản đồ phức tạp hoặc khi các điểm mục tiêu phân bố không đều.
- Ưu điểm của ACO:
 - ACO là một thuật toán metaheuristic dựa trên hành vi của đàn kiến, tìm kiếm giải pháp tối ưu toàn cục bằng cách sử dụng pheromone. Nhờ cơ chế cập nhật pheromone, thuật toán dần dần hội tụ về những hành trình tốt nhất sau nhiều lần lặp.
 - ACO đặc biệt hiệu quả trong việc giải bài toán TSP, vốn tương tự bài toán chọn thứ tự thăm các điểm mục tiêu trong ImpA*G.
- Tính bổ trợ:
 - ImpA* có ưu thế vượt trội trong việc tìm đường đi cục bộ an toàn và mượt mà giữa hai điểm bất kỳ, đảm bảo tránh chướng ngại vật và giảm số lần rẽ.
 - ACO mạnh trong việc tối ưu hóa thứ tự thăm các điểm mục tiêu, giúp giảm tổng độ dài hành trình và tránh các đường đi vòng vèo.
 - Kết hợp hai thuật toán này có thể tận dụng ưu điểm của cả hai: ImpA* đảm bảo đường đi cục bộ mượt mà và tránh chướng ngại vật, trong khi ACO tối

ưu hóa thứ tự thăm các điểm mục tiêu, khắc phục nhược điểm của chiến lược tham lam trong ImpA*G.

3.1.2 Thuật toán A* cải tiến

- *Tổng quan*

Thuật toán ImpA* được xây dựng dựa trên nền thuật toán A*, nhưng được cải tiến qua ba thành phần chính:

1. Hàm đánh giá nâng cao.
2. Tối ưu hướng mở rộng nút.
3. Tối ưu hóa quá trình làm mượt đường đi.

Mỗi thành phần góp phần tối ưu hiệu quả tìm kiếm và chất lượng đường đi.

- ❖ *Thiết kế hàm đánh giá nâng cao dựa trên thông tin chương ngại vật*

Thuật toán A* là một trong những thuật toán hiệu quả nhất để tìm đường ngắn nhất trong môi trường tĩnh. Hàm đánh giá truyền thống của A* được định nghĩa là:

$$F(n) = G(n) + H(n) \quad (3-1)$$

Trong phiên bản cải tiến, hàm đánh giá được điều chỉnh như sau:

$$F(n) = G(n) + H(n) + o(n) \quad (3-2)$$

Trong đó:

$$o(n) = -\alpha C(n) + \beta I(n) \quad (3-3)$$

$$C(n) = \frac{1}{50} \frac{R-r}{R} H(n) \quad (3-4)$$

$$I(n) = \frac{1}{20} \sum_{i=1}^M \frac{1}{\sqrt{(x_i - x_s)^2 + (y_i - y_s)^2}} \quad (3-5)$$

$$\alpha + \beta = 1 \quad (3-6)$$

Giải thích các thành phần:

- $C(n)$: là hệ số trọng số thể hiện mức độ ưu tiên tiến về phía điểm đích và là giá trị thông tin của nút tiếp theo xung quanh nút hiện tại đến đích. Nút càng gần đích thì giá trị $C(n)$ càng lớn, và tổng $H(n)$ càng nhỏ $\rightarrow$ giúp robot di chuyển

đúng hướng đích.

- r : là khoảng cách từ điểm hiện tại đến điểm đích tiếp theo.
- R : là khoảng cách tổng từ điểm bắt đầu đến đích.
- $I(n)$: là hàm dự đoán dựa trên thông tin về chướng ngại vật. M là số lượng chướng ngại vật dọc theo hướng di chuyển từ điểm hiện tại đến điểm đích. Càng gần vật cản, chi phí mở rộng nút càng lớn $\rightarrow$ khiến thuật toán tránh hướng đi qua vùng dày đặc vật cản. Cơ chế này giúp thuật toán định hướng tốt hơn từ sớm, tránh được các cực trị cục bộ và cải thiện hiệu quả tìm kiếm.
- α, β : là các giá trị trọng số.
 - Nếu $\beta = 0$: tức là không có vật cản nào nằm trên đường đi từ nút hiện tại đến điểm đích. Do đó, trọng số của hàm heuristic $H(n)$ có thể được tăng lên một cách thích hợp nhằm thu hẹp phạm vi tìm kiếm và nâng cao hiệu quả của quá trình tìm đường.
 - Nếu $\beta = 1$: tức là toàn bộ các vật cản đều nằm trên đường từ nút hiện tại đến điểm đích. Khi đó, trọng số của hàm $H(n)$ không được thay đổi, để mở rộng phạm vi tìm kiếm, tránh rơi vào cực trị cục bộ và đảm bảo thuật toán tìm được đường đi hợp lý hơn.
 - Các hệ số như $\frac{1}{50}$ và $\frac{1}{20}$ được lựa chọn để chuẩn hóa giá trị từng thành phần trong hàm đánh giá tổng thể, đảm bảo sự cân bằng giữa khoảng cách đến đích và mức độ ảnh hưởng của vật cản. Việc điều chỉnh các hệ số này cho phép kiểm soát trọng số của từng yếu tố, từ đó ảnh hưởng đến hướng tìm kiếm và chất lượng đường đi. Trong thực nghiệm, có thể tinh chỉnh các hệ số này để phù hợp với đặc thù môi trường và mục tiêu tối ưu cụ thể.

❖ Tối ưu hướng mở rộng nút

Thuật toán A* truyền thống thực hiện tìm kiếm mở rộng theo 8 hướng xung quanh mỗi nút hiện tại. Ưu điểm của việc mở rộng theo mọi hướng là giúp thuật toán thích ứng tốt với các môi trường có chướng ngại vật phức tạp. Tuy nhiên, nhược điểm là số

lượng hướng mở rộng lớn sẽ làm giảm hiệu suất tổng thể của quá trình tìm kiếm. Trong hầu hết các trường hợp thực tế, môi trường không quá phức tạp đến mức cần thiết phải mở rộng toàn bộ 8 hướng để hoàn thành bài toán lập kế hoạch đường đi.

Một phương pháp tối ưu hóa hướng mở rộng nút theo hướng thích ứng được đề xuất. Cụ thể, bước đầu tiên là xác định góc θ tạo thành bởi đường thẳng nối giữa nút hiện tại và điểm đích so với trục hoành (trục x). Góc θ chủ yếu được xác định dựa trên mối quan hệ giữa điểm bắt đầu, điểm đích và trục x. Góc này được tính theo công thức:

$$\theta = \arctan \frac{y_g - y_0}{x_g - x_0} \quad (3-7)$$

$$\theta = \begin{cases} 90^\circ & y_g > y_0 \cap x_g = x_0 \\ 270^\circ & y_g < y_0 \cap x_g = x_0 \\ \arctan \frac{y_g - y_0}{x_g - x_0} & y_g > y_0 \cup x_g \neq x_0 \\ 180^\circ + \arctan \frac{y_g - y_0}{x_g - x_0} & y_g < y_0 \cup x_g \neq x_0 \end{cases} \quad (3-8)$$

Bảng 3.1. Bảng quy tắc 3 hướng

θ	Giữ 3 hướng	Từ bỏ phương hướng
$[337.5^\circ, 360^\circ) \cup [0^\circ, 22.5^\circ)$	315 T, 000 T, 045 T	090 T, 135 T, 180 T, 225 T, 270 T
$[22.5^\circ, 67.5^\circ)$	000 T, 045 T, 090 T	135 T, 180 T, 225 T, 270 T, 315 T
$[67.5^\circ, 112.5^\circ)$	045 T, 090 T, 135 T	000 T, 180 T, 225 T, 270 T, 315 T
$[112.5^\circ, 157.5^\circ)$	090 T, 135 T, 180 T	045 T, 225 T, 270 T, 315 T, 000 T
$[157.5^\circ, 202.5^\circ)$	135 T, 180 T, 225 T	000 T, 045 T, 090 T, 270 T, 315 T
$[202.5^\circ, 247.5^\circ)$	180 T, 225 T, 270 T	000 T, 045 T, 090 T, 135 T, 315 T
$[247.5^\circ, 292.5^\circ)$	225 T, 270 T, 315 T	045 T, 090 T, 135 T, 180 T, 000 T
$[292.5^\circ, 337.5^\circ)$	270 T, 315 T, 000 T	045 T, 090 T, 135 T, 180 T, 225 T

Bảng 3.2. Bảng quy tắc 5 hướng

θ	Giữ 5 hướng	Từ bỏ phương hướng
----------	-------------	--------------------

[337. 5°, 360°) $\cup$ [0°, 22. 5°)	000 T, 045 T, 090 T, 270 T, 315 T	135 T, 180 T, 225 T
[22. 5°, 67. 5°)	000 T, 045 T, 090 T, 135 T, 315 T	180 T, 225 T, 270 T
[67. 5°, 112. 5°)	000 T, 045 T, 090 T, 135 T, 180 T	225 T, 270 T, 315 T
[112. 5°, 157. 5°)	045 T, 090 T, 135 T, 180 T, 225 T	270 T, 315 T, 000 T
[157. 5°, 202. 5°)	090 T, 135 T, 180 T, 225 T, 270 T	000 T, 045 T, 315 T
[202. 5°, 247. 5°)	135 T, 180 T, 225 T, 270 T, 315 T	000 T, 045 T, 090 T
[247. 5°, 292. 5°)	180 T, 225 T, 270 T, 315 T, 000 T	045 T, 090 T, 135 T
[292. 5°, 337. 5°)	225 T, 270 T, 315 T, 000 T, 045 T	090 T, 135 T, 180 T

Trong phiên bản cải tiến, thuật toán A* không còn mở rộng theo đủ 8 hướng tại mỗi bước như cách truyền thống. Thay vào đó, hướng mở rộng được điều chỉnh một cách thích ứng dựa trên góc giữa điểm hiện tại và điểm đích, cũng như mật độ vật cản xung quanh.

Cụ thể, tại mỗi bước, thuật toán sẽ:

- Tính góc định hướng di chuyển (θ): xác định hướng từ điểm hiện tại đến điểm đích bằng cách tính góc giữa vector kết nối hai điểm và trục hoành. Góc này được chia thành 8 vùng (mỗi vùng 45°), tương ứng với 8 hướng di chuyển chính trên lưới.
- Đếm số lượng vật cản xung quanh: thuật toán khảo sát trong vùng quét hình tròn có bán kính $r = \sqrt{5}$ quanh điểm hiện tại. Dựa vào số lượng vật cản trong

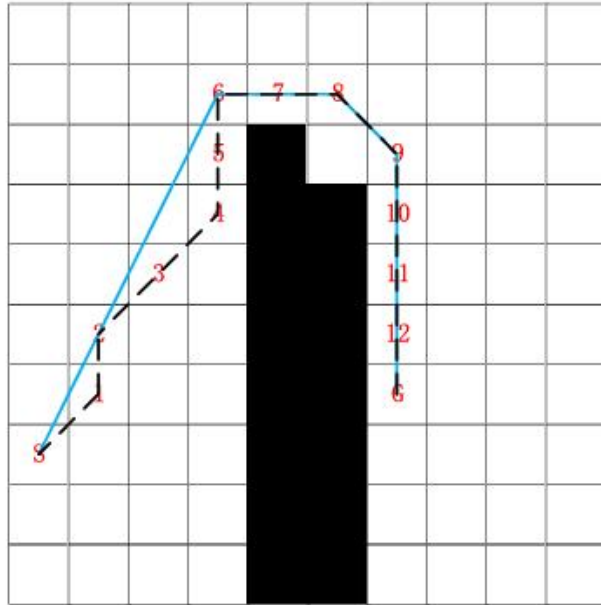
vùng này, nó quyết định phạm vi mở rộng:

- Không có vật cản: chỉ mở rộng theo 3 hướng ưu tiên, phù hợp với hướng θ đang hướng đến.
 - Có ít vật cản (dưới 4 vật): mở rộng theo 5 hướng, giúp tránh vật cản cục bộ trong khi vẫn giữ được tính ưu tiên.
 - Nhiều vật cản: trở về mở đủ 8 hướng như A* truyền thống để đảm bảo khả năng tìm được đường tránh chướng ngại vật.
- Chọn hướng mở rộng phù hợp: dựa vào giá trị góc θ vừa tính, thuật toán tra bảng quy tắc để chọn nhóm hướng phù hợp (3 hoặc 5 hướng). Các hướng này được ưu tiên đưa vào danh sách mở rộng, giúp giảm số lượng nút mở rộng không cần thiết và tăng tốc độ tìm kiếm.

❖ Tối ưu hóa đường đi

Sau khi tìm được đường đi an toàn bằng thuật toán A* cải tiến, kết quả thu được thường là chuỗi các điểm lưới liên tiếp, trong đó tồn tại nhiều điểm trung gian dư thừa không thực sự cần thiết để định hình hướng di chuyển. Những điểm này gây nên nhiều góc rẽ không cần thiết, làm giảm độ trơn mượt và tăng tổng chiều dài đường đi.

Nguyên lý của thuật toán tối ưu hóa được minh họa trong hình 2.1. Ví dụ, đường đi do A* truyền thống tạo ra là chuỗi điểm S,1,2,3,4,5,6,7,8,9,10,11,12,13,G – trong đó có rất nhiều nút không cần thiết. Thuật toán Floyd giữ lại các nút chính của đường đi và loại bỏ các nút gây dư thừa, giúp tạo ra một đường đi ngắn hơn và mượt hơn, nhưng vẫn đảm bảo không va chạm vật cản.



Hình 3.1. Minh họa tối ưu hóa làm mịn đường đi

Các bước làm mượt đường:

- Bước 1: xuất ra đường đi ban đầu được tạo bởi thuật toán A* cải tiến (chưa tối ưu).
- Bước 2: duyệt qua toàn bộ các nút và xác định điểm gãy (inflection point). Kiểm tra xem độ dốc k_1 và k_2 có bằng nhau không theo công thức (10) và (11):

$$k_1 = \frac{y_i - y_{i-1}}{x_i - x_{i-1}} \quad (3-9)$$

$$k_2 = \frac{y_{i+1} - y_i}{x_{i+1} - x_i} \quad (3-10)$$

Nếu $k_1 \neq k_2$, thì điểm (x_i, y_i) là điểm gãy. Thuật toán chỉ giữ lại các điểm sau:

- Điểm bắt đầu S
- Điểm kết thúc G
- Các điểm gãy được xác định theo công thức trên

Các điểm còn lại được gán theo tọa độ mới (x_n, y_n) để phân biệt rõ các điểm gãy

- Bước 3: lặp qua điểm bắt đầu và các điểm uốn. Đặt 3 điểm uốn liên tiếp (x_{n-1}, y_{n-1}) , (x_n, y_n) , và (x_{n+1}, y_{n+1}) thành một nhóm, và xác định mối quan hệ giữa đường đi D giữa điểm uốn đầu tiên và thứ ba với đường đi d giữa ba điểm uốn. D và d được thể hiện trong công thức (12) và (13). Nếu $d \leq D$ và d không đi qua chướng ngại vật, thì điểm đầu tiên và thứ 3 được giữ lại, ngược lại, ba điểm sẽ được giữ lại.

$$D = \sqrt{(x_{n+1} - x_{n-1})^2 + (y_{n+1} - y_{n-1})^2} \quad (3-11)$$

$$d = \sqrt{(x_{n-1} - x_n)^2 + (y_{n-1} - y_n)^2} + \sqrt{(x_{n+1} - x_n)^2 + (y_{n+1} - y_n)^2} \quad (3-12)$$

- Bước 4: trích xuất các nút còn lại, xuất ra đường đi được tối ưu hóa. Thuật toán kết thúc.

Tính góc rẽ trong quy hoạch đường đi được xác định bằng cách sử dụng công thức:

$$\text{angle} = \arccos\left(\frac{(x_{n-1} - x_n)(x_{n+1} - x_n) + (y_{n-1} - y_n)(y_{n+1} - y_n)}{\sqrt{(x_{n-1} - x_n)^2 + (y_{n-1} - y_n)^2} \sqrt{(x_{n+1} - x_n)^2 + (y_{n+1} - y_n)^2}}\right) \quad (3-13)$$

Tổng thể, ba thành phần trên tạo thành một thuật toán A* cải tiến vừa hiệu quả về thời gian, vừa tối ưu về chất lượng đường đi, và đóng vai trò làm nền tảng cho các biến thể kết hợp trình bày trong các mục tiếp theo.

- Ưu và nhược điểm

- *Ưu điểm*

- Giảm số lượng hướng mở rộng: thay vì mở 8 hướng cố định, A* cải tiến sử dụng chiến lược chọn hướng thích ứng theo góc định hướng và mật độ vật cản, giúp giảm số lượng nút cần duyệt, tránh lãng phí tài nguyên vào các hướng không khả thi.
- Tối ưu hóa đường đi bằng đánh giá thích nghi: hàm đánh giá cải tiến giúp cân bằng giữa mục tiêu tiến gần đến đích, tránh xa khu vực có nhiều vật cản, điều chỉnh linh hoạt theo môi trường.
- Làm mượt đường đi hiệu quả: việc loại bỏ các điểm gãy không cần thiết bằng thuật toán Floyd giúp giảm tổng độ dài hành trình, tạo đường đi trơn mượt, ít rẽ.

- *Nhược điểm*

- Chưa xử lý bài toán đa mục tiêu: A* cải tiến chỉ tìm đường từ một, điểm đầu đến một đích duy nhất. Khi có nhiều mục tiêu trung gian, A* cải tiến cần được lặp lại nhiều lần, dẫn đến thứ tự ghé đích không tối ưu.
- Phụ thuộc vào tham số điều chỉnh (α, β) : việc tính toán trọng số trong hàm đánh giá cần được thiết kế cẩn thận để tránh mất cân bằng giữa hướng đích và tránh vật cản.

- Vẫn mang bản chất heuristic nên không đảm bảo tối ưu toàn cục tuyệt đối, nhất là trong các môi trường có mật độ vật cản cao hoặc cấu trúc phức tạp.

Tóm lại, A* cải tiến là một phiên bản cải tiến hiệu quả của A*, đặc biệt phù hợp trong môi trường đơn mục tiêu, tĩnh và có cấu trúc lưới rõ ràng. Tuy nhiên, khi mở rộng sang bài toán đa mục tiêu, A* cải tiến gặp hạn chế về khả năng tối ưu thứ tự ghé thăm và hiệu suất tính toán. Do đó, hướng tiếp cận tiềm năng là kết hợp A* cải tiến với thuật toán ACO, trong đó A* đảm nhiệm tìm đường ngắn giữa các cặp điểm, còn ACO tối ưu thứ tự ghé thăm các điểm trung gian. Sự kết hợp này cho phép tận dụng ưu điểm của cả hai thuật toán, nâng cao hiệu quả trong môi trường phức tạp, nhiều mục tiêu.

3.1.3 Thuật toán kết hợp A* cải tiến và ACO

Thuật toán ImpA*ACO được xây dựng dựa trên sự kết hợp hai thành phần:

- ACO để xác định thứ tự tối ưu ghé thăm các điểm mục tiêu trung gian (giải quyết bài toán tương đương với TSP).
- ImpA* để tìm đường đi tốt nhất giữa các cặp điểm theo thứ tự đó, đảm bảo an toàn và mượt mà trong môi trường có chướng ngại vật.

Quá trình thực hiện được chia thành ba giai đoạn chính:

- **Giai đoạn tiền xử lý bằng thuật toán ImpA*:**
 - Thuật toán ImpA* được sử dụng để tính toán trước khoảng cách giữa tất cả các cặp điểm: điểm xuất phát, các điểm trung gian và điểm đích
 - Kết quả tạo thành một ma trận khoảng cách, làm đầu vào cho thuật toán ACO ở bước sau
- **Giai đoạn tối ưu thứ tự ghé đích bằng thuật toán ACO:**
 - Mỗi cá thể kiến sinh ra một hành trình bằng cách chọn thứ tự ghé điểm dựa trên xác suất.
 - Xác suất chọn điểm tiếp theo phụ thuộc vào dấu vết pheromone và khoảng cách.
 - Sau mỗi vòng lặp, pheromone được cập nhật dựa trên chất lượng hành trình.

- Sau nhiều lần lặp, thuật toán chọn được thứ tự ghé điểm tối ưu (hoặc gần tối ưu nhất).

- **Giai đoạn lập kế hoạch chi tiết bằng $ImpA^*$:**

Sau khi có thứ tự ghé đích do ACO quyết định, thuật toán sẽ:

- Lần lượt kết nối các điểm theo thứ tự được chọn bằng $ImpA^*$.
- Từng đoạn đường giữa các điểm đều được tối ưu hóa riêng biệt để: tránh chướng ngại vật, giảm số điểm rẽ, làm mượt đường đi.
- Ghép nối các đoạn đường lại để hình thành hành trình tổng thể.

c

Hình 3.2. Lưu đồ thuật toán A^* cải tiến kết hợp ACO

Mã giả thuật toán $ImpA^*ACO$

Input: start: điểm bắt đầu; targets: tập các điểm mục tiêu trung gian; goal: điểm đích; grid: bản đồ lưới (0: trống, 1: vật cản)

Output: full_path: đường đi hoàn chỉnh từ start $\rightarrow$ targets $\rightarrow$ goal

1. Tập tất cả điểm cần đi qua:

points $\leftarrow$ [start] + targets + [goal]

2. Tính chi phí thực tế giữa các cặp điểm:

dist_matrix, path_lookup $\leftarrow$ compute_distance_matrix(points, grid)

3. Tìm thứ tự tối ưu ghé đích bằng ACO:

route_indices $\leftarrow$ ACO_TSP(dist_matrix)

4. Duyệt theo thứ tự ACO tìm được, tạo đường đi giữa từng cặp điểm:

full_path $\leftarrow$ []

target_order $\leftarrow$ []

For i = 0 to length(route_indices) - 2:

u $\leftarrow$ route_indices[i]

v $\leftarrow$ route_indices[i + 1]

$path_{uv} \leftarrow path_lookup[(u, v)]$

Nếu $full_path$ rỗng:

$full_path \leftarrow path_{uv}$

Ngược lại:

$full_path \leftarrow full_path + path_{uv}$ (bỏ điểm đầu nếu trùng)

Nếu v là mục tiêu trung gian:

thêm v vào $target_order$

5. Làm mượt đường đi:

$full_path \leftarrow smooth_path(full_path, grid, must_include=[start, goal] + targets)$

6. Trả kết quả:

return $full_path, target_order$

3.1.4 Ưu và nhược điểm

- *Ưu điểm*

- Tối ưu được hành trình toàn cục: nhờ sử dụng thuật toán ACO để xác định thứ tự ghé đích, ImpA*ACO có khả năng tìm được thứ tự thăm các điểm mục tiêu tối ưu (hoặc gần tối ưu), giúp giảm tổng chiều dài hành trình so với chiến lược tham lam (ImpA*G)
- Kết hợp hiệu quả giữa tối ưu cục bộ và toàn cục: ImpA* đảm bảo chất lượng tốt cho từng đoạn đường (tối ưu cục bộ), trong khi ACO giải quyết bài toán sắp xếp điểm mục tiêu (tối ưu toàn cục). Sự kết hợp này tạo nên hành trình vừa ngắn, vừa mượt, an toàn và thực tế.
- Hiệu quả trong bản đồ phức tạp: trong các bản đồ có nhiều vật cản hoặc khi các điểm mục tiêu phân bố không đều, ImpA*ACO giúp robot tránh các hành trình vòng vèo không cần thiết mà ImpA*G có thể gặp phải.
- Khả năng cải tiến qua lặp: ACO là thuật toán metaheuristic, nên chất lượng lời giải có thể được cải thiện dần theo số vòng lặp và số lượng kiến. Điều này tạo tính linh hoạt trong việc điều chỉnh độ chính xác – tốc độ tùy theo nhu cầu.

- *Nhược điểm*

- Chi phí tính toán cao hơn ImpA*G: thuật toán ACO cần chạy qua nhiều vòng lặp và tính toán xác suất cho từng cặp điểm, dẫn đến thời gian xử lý cao hơn so với ImpA*G. Điều này có thể không phù hợp cho hệ thống có tài nguyên hạn chế hoặc yêu cầu phản hồi siêu nhanh.
- Phụ thuộc vào tham số của ACO: chất lượng lời giải phụ thuộc vào các tham số như: số lượng kiến, hệ số bốc hơi pheromone, số vòng lặp... Nếu tham số không phù hợp, thuật toán dễ hội tụ chậm hoặc bị kẹt ở cực trị cục bộ.

CHƯƠNG 4. CÀI ĐẶT THỰC NGHIỆM

4.1 Cài đặt môi trường

Để đảm bảo quá trình phát triển, thử nghiệm và đánh giá các thuật toán diễn ra hiệu quả, đề tài được triển khai trong môi trường phần cứng và phần mềm phù hợp, với các công cụ hỗ trợ mạnh mẽ trong việc mô phỏng, xử lý dữ liệu và trực quan hóa kết quả.

- **Cấu hình phần cứng**

Các thực nghiệm được thực hiện trên máy tính cá nhân với cấu hình như sau:

- Tên máy: Modern 14 B11MOU.
- Hệ điều hành: Windows 11 64-bit.
- Bộ vi xử lý: 11th Gen Intel(R) Core(TM) i7-1195G7 @ 2.90GHz, 2918 Mhz, 4 Core(s), 8 Logical Processor(s).
- Bộ nhớ RAM: 8GB.
- Ổ cứng: SSD 256GB.
- Card đồ họa: Intel Iris Xe Graphics (tích hợp trong CPU).

Do các thuật toán trong đề tài không yêu cầu tính toán chuyên sâu về đồ họa hay học sâu, cấu hình trên hoàn toàn đáp ứng được nhu cầu mô phỏng và đánh giá hiệu quả của mô hình.

- **Phần mềm và thư viện lập trình**

- Ngôn ngữ lập trình: Python phiên bản 3.10.4
- Công cụ lập trình: Pycharm
- Thư viện đã sử dụng:
 - **NumPy**: Thư viện xử lý mảng và toán học hiệu năng cao. Được sử dụng để quản lý dữ liệu bản đồ dạng lưới, xử lý vector, tính khoảng cách, ma trận trọng số.
 - **Matplotlib**: Dùng để trực quan hóa bản đồ, các đường đi, điểm bắt đầu/kết thúc và các mục tiêu trung gian. Đồng thời hỗ trợ tạo giao diện tương tác người dùng

bằng cách kết hợp với PyQt.

- **PyQt5**: Thư viện tạo giao diện đồ họa (GUI). Được sử dụng để xây dựng giao diện ứng dụng với các chức năng như tạo bản đồ, chọn thuật toán, hiển thị đường đi, bảng so sánh kết quả.
- **heapq**: Thư viện chuẩn của Python hỗ trợ hàng đợi ưu tiên, dùng trong việc triển khai thuật toán A* và các biến thể để lựa chọn node tiếp theo hiệu quả.
- **math**: Cung cấp các hàm toán học như hypot, atan2, cos, sin, dùng trong tính khoảng cách Euclidean, góc rẽ.
- **random**: Hỗ trợ tạo bản đồ ngẫu nhiên và lựa chọn xác suất trong thuật toán ACO.
- **time**: Dùng để đo thời gian thực thi của từng thuật toán.
- **os**: Dùng trong thao tác với tệp bản đồ (nạp/lưu dữ liệu bản đồ từ file .npy).

4.2 Các tiêu chí đánh giá

Để đánh giá hiệu quả của các thuật toán đề xuất trong việc lập kế hoạch đường đi cho robot di động, đồ án sử dụng các tiêu chí định lượng cụ thể, phản ánh các khía cạnh quan trọng trong quá trình di chuyển của robot. Các tiêu chí này bao gồm:

- *Tổng độ dài đường đi*:
 - Là tổng quãng đường mà robot phải di chuyển để hoàn thành nhiệm vụ từ điểm xuất phát, đi qua các mục tiêu trung gian và đến đích cuối cùng. Đo bằng đơn vị ô lưới trong bản đồ 2D. Đường đi càng ngắn càng tốt, nhằm tiết kiệm thời gian và năng lượng. Tổng độ dài đường đi qua nhiều điểm mục tiêu của được tính theo công thức sau:

$$L_{path} = \left(\sum_{j=0}^{n-1} \sum_{i=1}^s \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \right) \quad (4-1)$$

- *Số lần rẽ nhánh*:
 - Là số điểm mà tại đó hướng di chuyển của robot thay đổi, tức là robot cần quay hoặc đổi hướng để tiếp tục hành trình. Số lần rẽ càng ít càng tốt, để đảm bảo hành trình mượt mà, ổn định. Được tính bằng cách so sánh hai vector liên tiếp:

$$\vec{v_1} = (x_n - x_{n-1}, y_n - y_{n-1}), \vec{v_2} = (x_{n+1} - x_n, y_{n+1} - y_n) \quad (4-2)$$

Nếu $\vec{v_1} \neq \vec{v_2}$ thì tại điểm đó có một lần rẽ nhánh.

- *Tổng góc quay:*
 - Là tổng các góc mà robot phải quay trong toàn bộ hành trình, tính theo độ ($^{\circ}$). Phản ánh độ gấp khúc của đường đi, càng nhiều góc lớn $\rightarrow$ càng giảm tính ổn định khi vận hành. Tổng góc quay nhỏ hơn thể hiện đường đi trơn mượt hơn, được tính theo công thức (3-13)
- *Hàm fitness*
 - Là một chỉ số đánh giá tổng thể mức độ hiệu quả của từng thuật toán, được tính toán dựa trên 3 tiêu chí trên. Các trọng số phản ánh tầm quan trọng của từng tiêu chí. Giá trị fitness càng thấp thì thuật toán càng tốt, vì biểu thị đường đi ngắn hơn, ít rẽ hơn và mượt hơn. Công thức tính điểm fitness trong đề án:

$$Fitness = 0.4L + 0.3A + 0.3I \quad (4-3)$$

Trong đó:

- L (Length): tổng độ dài đường đi của robot.
- A (Angle): tổng góc quay của robot trong hành trình
- I (Inflection): số lần rẽ hướng trong hành trình.

4.3 Kịch bản thực nghiệm

Để đánh giá khách quan hiệu quả của các thuật toán được đề xuất trong quá trình lập kế hoạch đường đi cho robot, đề án xây dựng một loạt kịch bản kiểm thử mô phỏng dựa trên môi trường lưới 2D với các cấu hình khác nhau. Mỗi kịch bản bao gồm bản đồ, điểm xuất phát, điểm đích, và một tập hợp các điểm mục tiêu trung gian. Các kịch bản được thiết kế nhằm kiểm tra khả năng của thuật toán trong các tình huống điển hình, từ đơn giản đến phức tạp.

- **Thực nghiệm 1:** So sánh hiệu suất các thuật toán tìm đường:
 - **Case 1:** So sánh toàn diện giữa các thuật toán
 - Chiến lược: Tạo bản đồ với cùng kích thước, cùng điểm xuất phát,

cùng điểm đích và mục tiêu trung gian. Các thuật toán sẽ được áp dụng lần lượt trên cùng một bản đồ

- Mục tiêu: So sánh trực tiếp chất lượng của từng thuật toán khi áp dụng cho cùng một môi trường, từ đó rút ra những đặc điểm mạnh yếu của từng thuật toán.
- Số lượng bản đồ: 10.

- **Thực nghiệm 2:** So sánh dựa trên sự ảnh hưởng của môi trường:

- **Case 2:** Ảnh hưởng của mật độ chướng ngại vật

- Chiến lược: Tạo các bản đồ với mật độ chướng ngại vật khác nhau, giữ nguyên vị trí xuất phát, đích đến và mục tiêu trung gian.
 - Mục tiêu: Phân tích ảnh hưởng của độ phức tạp bản đồ đến khả năng tìm đường của thuật toán
 - Số lượng bản đồ: 10.

- **Case 3:** Ảnh hưởng của số lượng đích đến

- Chiến lược: Tạo các bản đồ với số lượng mục tiêu tăng dần, giữ nguyên bản đồ và vật cản.
 - Mục tiêu: Đánh giá khả năng lập lịch ghé đích của các thuật toán.
 - Số lượng bản đồ: 10.

- **Case 4:** Ảnh hưởng của hình dạng chướng ngại vật

- Chiến lược: Thiết kế bản đồ có vật cản mang hình dạng đặc biệt như hình chữ cái, ngõ cụt.
 - Mục tiêu: Kiểm tra khả năng điều hướng khi gặp các chướng ngại vật không gian hoặc cấu trúc gây nhiễu.
 - Số lượng bản đồ: 10.

- **Case 5:** Đánh giá dựa trên mô phỏng môi trường thực tế

- Chiến lược: Xây dựng các bản đồ mô phỏng theo bố cục thực tế

(thành phố, nhà kho, không gian trong nhà).

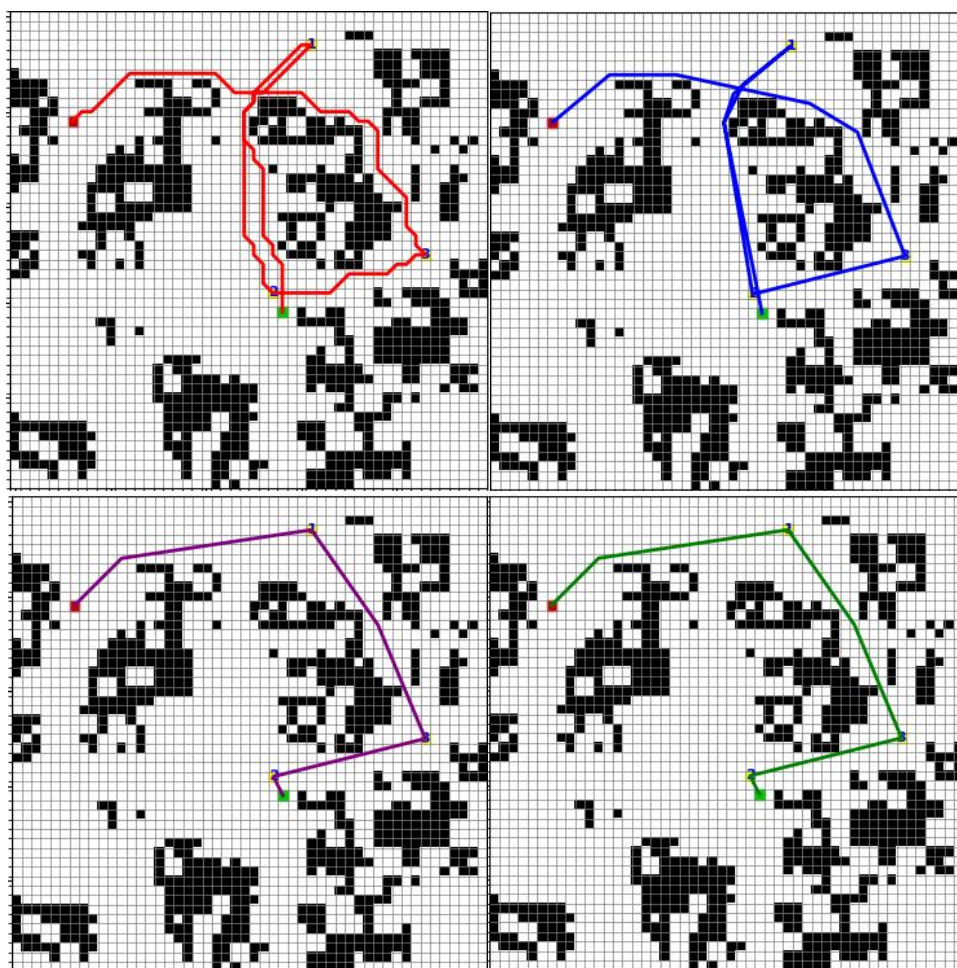
- Mục tiêu: Kiểm tra khả năng ứng dụng thực tế của thuật toán trong các môi trường có bố trí thực tế, nhiều hướng đi, vật cản logic.
- Số lượng bản đồ: 3.

4.4 Kết quả thực nghiệm

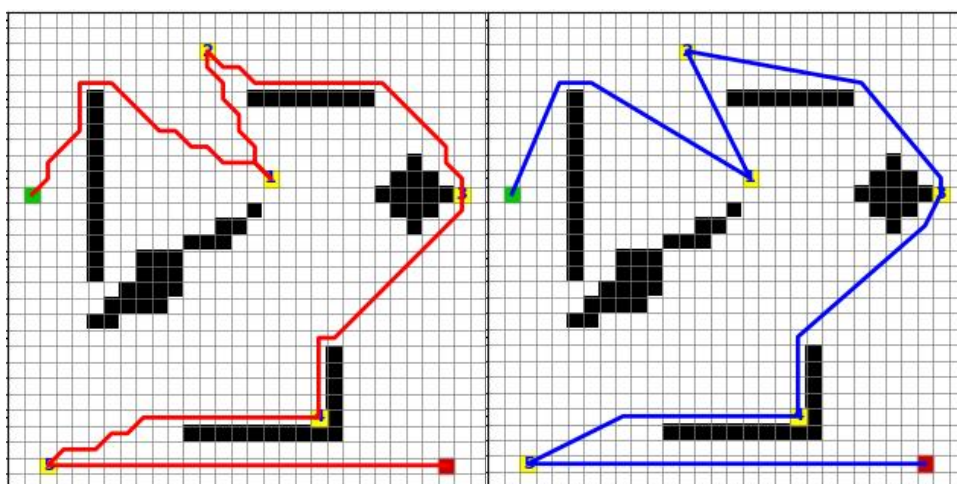
Bảng 4.1 *Chú thích giao diện khi chạy thực nghiệm*

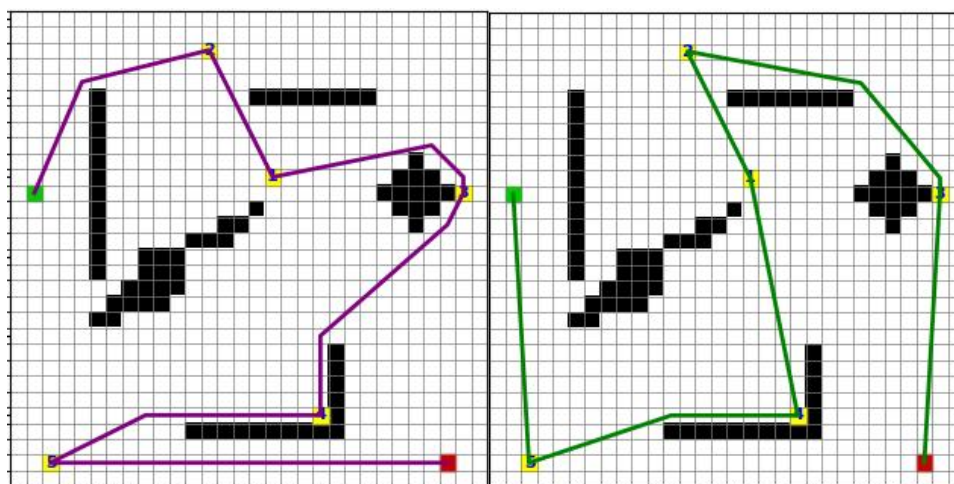
Điểm bắt đầu	
Điểm đích	
Mục tiêu trung gian	
Chướng ngại vật	
Đường đi A*	
Đường đi ImpA*	
Đường đi ImpA*G	
Đường đi ImpA*ACO	

4.4.1 Case 1: So sánh toàn diện giữa các thuật toán

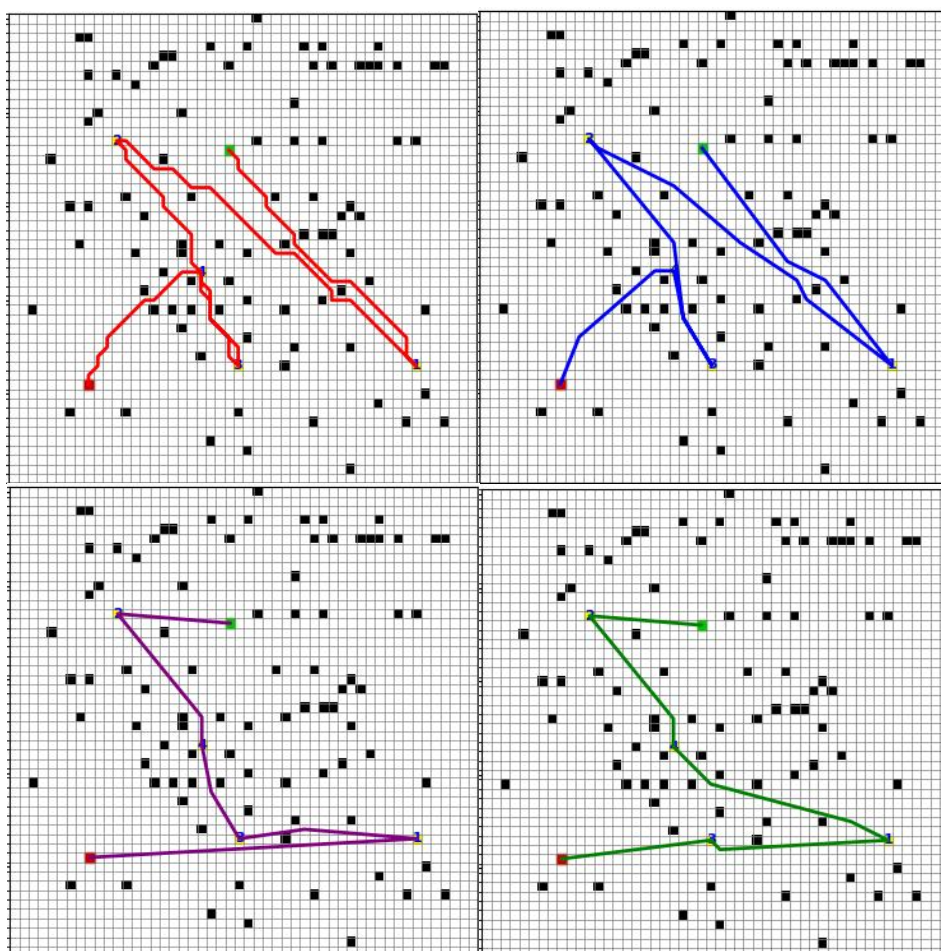


Bộ dữ liệu 4.1b





Bộ dữ liệu 4.1c



Bộ dữ liệu 4.1f

Hình 4.1. Kết quả thực nghiệm case 1

Bảng 4.2 Kết quả thực nghiệm case 1

Môi	Thuật toán	Độ dài	Tổng số	Số lần rẽ	Fitness
-----	------------	--------	---------	-----------	---------

trường		đường đi (m)	góc quay (°)		
Bộ dữ liệu 4.1a (15 × 15)	A*	48.6	810	13	266.323
	ImpA*	47.3	450	6	155.719
	ImpA*G	47.3	450	6	155.719
	ImpA*ACO	47.3	450	6	155.719
Bộ dữ liệu 4.1b (50 × 50)	A*	133.2	2295	46	755.562
	ImpA*	125.9	601.5	11	234.100
	ImpA*G	71.2	313.5	5	124.040
	ImpA*ACO	71.2	313.5	5	124.040
Bộ dữ liệu 4.1c (30 × 30)	A*	114.8	2115	37	691.536
	ImpA*	110.8	819.5	12	293.761
	ImpA*G	99.4	673.9	11	245.239
	ImpA*ACO	95.8	450	8	175.711
Bộ dữ liệu 4.1d (40 × 40)	A*	141	3690	38	1179.605
	ImpA*	136.2	2666.5	54	865.825
	ImpA*G	126.1	2413.1	34	784.552
	ImpA*ACO	103.8	1876.7	31	613.824
Bộ dữ liệu 4.1e (20 × 20)	A*	109.3	1980	33	647.602
	ImpA*	106.2	1245.8	17	421.316
	ImpA*G	73.2	932.2	13	312.859
	ImpA*ACO	72.9	931.5	13	312.525
Bộ dữ liệu 4.1f (50 × 50)	A*	134.8	2565	48	837.801
	ImpA*	128.4	889.4	16	322.960
	ImpA*G	94.4	456.3	7	176.741
	ImpA*ACO	90.5	510.6	8	191.771

	A*	153.9	2520	45	831.070
Bộ dữ liệu 4.1g	ImpA*	145.6	1002.7	16	363.839
45 × 45)	ImpA*G	130.8	653.9	12	252.107
	ImpA*ACO	129.8	658.8	14	253.743
	A*	89	1530	27	502.717
Bộ dữ liệu 4.1h	ImpA*	85.5	740.1	10	259.246
(30 × 30)	ImpA*G	61.1	481.3	7	170.925
	ImpA*ACO	61.1	481.3	7	170.925
	A*	200.3	2745	47	917.702
Bộ dữ liệu 4.1i	ImpA*	194.2	2033.1	29	696.331
(30 × 30)	ImpA*G	127.7	1271	18	437.780
	ImpA*ACO	116.3	1060.4	18	370.033
	A*	101.3	1845	31	603.330
Bộ dữ liệu 4.1j	ImpA*	99.9	1341.9	24	449.733
(30 × 30)	ImpA*G	87.9	1168.2	23	392.534
	ImpA*ACO	62.8	1018.7	18	336.125

Kết quả từ bảng 4.1 cho thấy sự khác biệt rõ ràng giữa các thuật toán về hiệu quả lập kế hoạch đường đi:

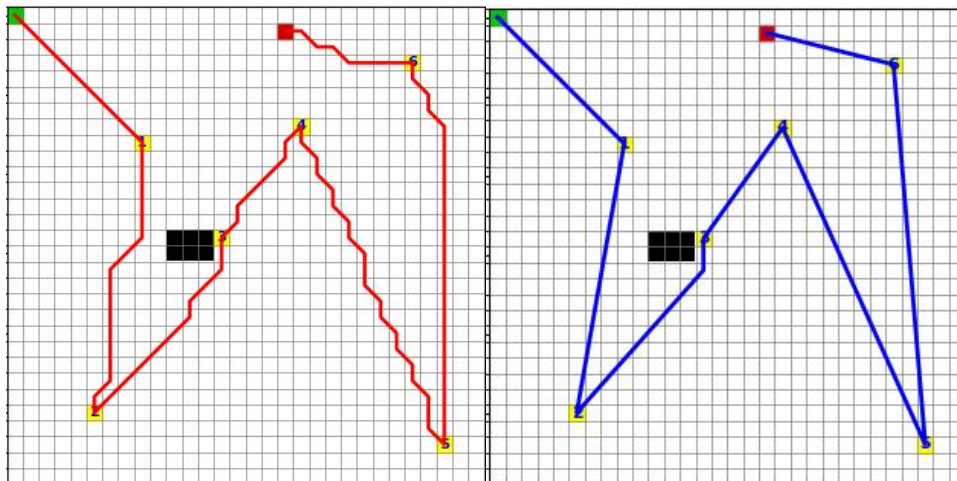
- Về độ dài đường đi, thuật toán A* truyền thống luôn cho kết quả lớn nhất ở hầu hết các bản đồ kiểm thử. Ngược lại, các phiên bản cải tiến như ImpA*, ImpA*G và ImpA*ACO đều giúp rút ngắn quãng đường di chuyển một cách đáng kể. Hiệu quả này đặc biệt rõ rệt trong các bản đồ có kích thước lớn hoặc nhiều đích đến. Chẳng hạn, ở bản đồ 4.1f (50×50), A* tạo ra đường đi dài 134.8m, trong khi ImpA*ACO chỉ cần 90.5m — giảm hơn 44.3m, tương đương hơn 30%.
- Xét về độ mượt của hành trình, được phản ánh qua tổng góc quay và số lần rẽ,

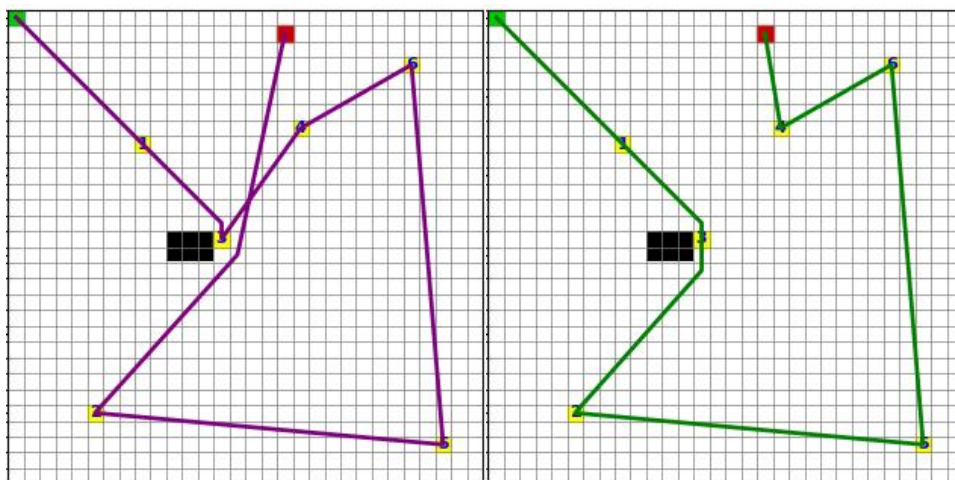
thuật toán A* cho kết quả không khả quan. Các đường đi do A* sinh ra có nhiều đoạn gấp khúc với tổng góc quay rất cao, lên tới hơn $2000^\circ - 3000^\circ$ ở nhiều bản đồ. Trong khi đó, các thuật toán cải tiến đã giúp giảm hơn một nửa số lần rẽ và góc quay.

- Về chỉ số fitness tổng hợp – đại diện cho hiệu quả toàn diện của đường đi – A* luôn có giá trị cao nhất trong tất cả các bản đồ, cho thấy hiệu suất tổng thể thấp nhất. Ngược lại, các thuật toán cải tiến có fitness thấp hơn đáng kể, chứng minh khả năng tạo ra đường đi ngắn hơn, ít rẽ và hiệu quả hơn.

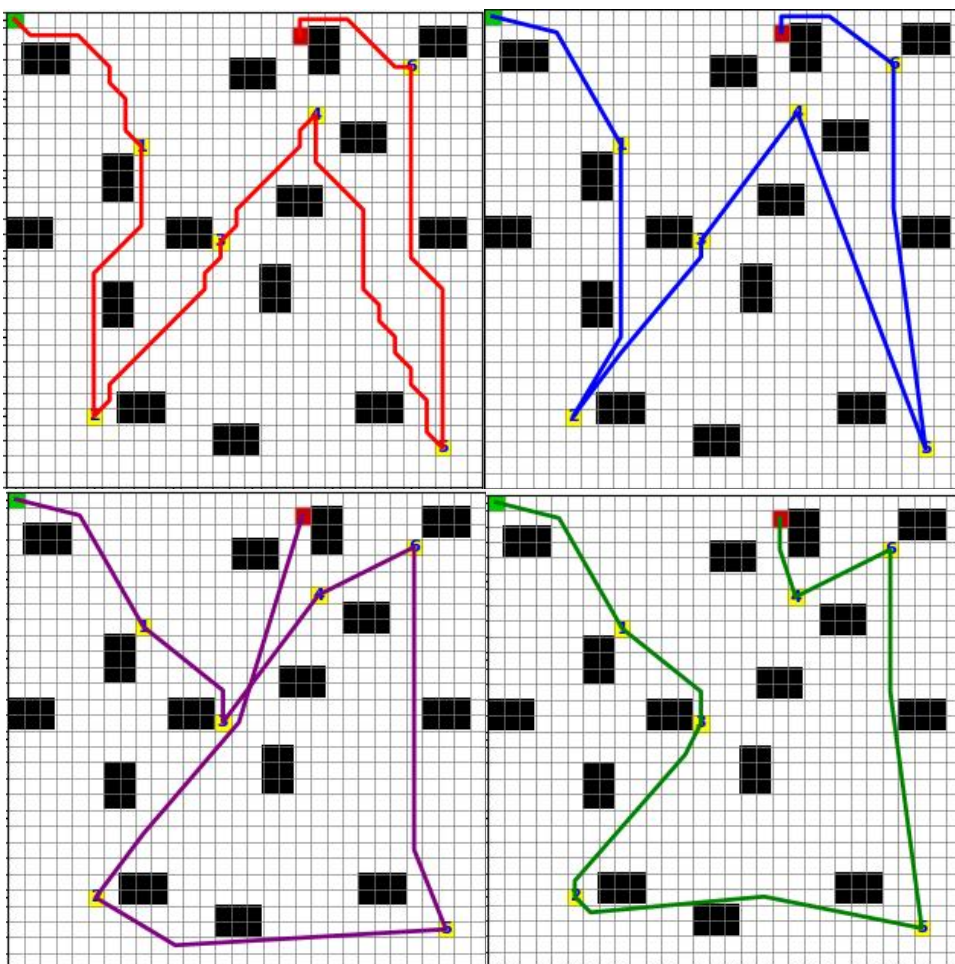
Từ đó, có thể nhận định rằng thuật toán A* truyền thống cho kết quả thấp nhất về mọi mặt: đường đi dài, nhiều khúc rẽ, hành trình kém mượt, fitness cao, không phù hợp để giải bài toán lập kế hoạch đường đi đa mục tiêu trong môi trường phức tạp. ImpA* mang lại cải tiến rõ rệt về cả độ dài và độ ổn định, tuy nhiên vì không xét đến thứ tự ghé đích, ImpA* vẫn có thể gặp giới hạn trong các bản đồ có nhiều mục tiêu phân bố không đều, trong khi ImpA*G tận dụng tính tham lam để tối ưu thời gian tìm kiếm và độ dài, nhưng hiệu quả chưa vượt trội trong môi trường đơn giản. Cuối cùng, ImpA*ACO là thuật toán ổn định và toàn diện nhất, đặc biệt hiệu quả khi áp dụng vào môi trường có nhiều đích đến hoặc cấu trúc bản đồ phức tạp.

4.4.2 Case 2: Ảnh hưởng của mật độ chướng ngại vật





Bộ dữ liệu 4.2a



Bộ dữ liệu 4.2c



Bộ dữ liệu 4.2i

Hình 4.2. Kết quả thực nghiệm case 2

Bảng 4.3. Kết quả thực nghiệm case 2

Môi trường	Mật độ	Thuật toán	Độ dài đường đi (m)	Tổng số góc quay (°)	Số lần rẽ	Fitness
Bộ dữ liệu 4.2a (30× 30)	1% (9/900)	A*	110.3	2205	42	718.230
		ImpA*	105.5	632.5	7	234.050
		ImpA*G	110	585.8	8	222.139
		ImpA*ACO	93.7	538.3	8	201.386
Bộ dữ liệu 4.2b	5% (45/900)	A*	115.3	1935	32	636.230
		ImpA*	111.2	942.3	12	330.800

(30× 30)		ImpA*G	116.8	963.7	14	340.040
		ImpA*ACO	99.9	922.3	15	321.172
Bộ dữ liệu 4.2c (30 × 30)	10% (90/900)	A*	114.5	2115	39	691.999
		ImpA*	109.4	837.2	13	298.846
		ImpA*G	112.4	693.1	12	256.480
		ImpA*ACO	93.1	662.9	14	240.303
Bộ dữ liệu 4.2d (30 × 30)	15% (135/900)	A*	131.2	2475	44	808.1905
		ImpA*	126.3	1272.8	20	438.368
		ImpA*G	114.6	1012.2	17	354.590
		ImpA*ACO	95.7	814.1	16	287.327
Bộ dữ liệu 4.2e (30 × 30)	20% (180/900)	A*	124.5	3195.0	52	1023.899
		ImpA*	118.3	1547.8	19	517.332
		ImpA*G	132.2	1927.1	26	638.808
		ImpA*ACO	111.8	1789.8	20	587.651
Bộ dữ liệu 4.2f (30 × 30)	25% (225/900)	A*	148.1	3420	50	1100.245
		ImpA*	142.6	2227.5	22	731.911
		ImpA*G	128.8	1332.6	22	457.922
		ImpA*ACO	114	1791.6	22	589.663
Bộ dữ liệu 4.2g 30 × 30)	30% (270/900)	A*	156.5	2970	38	964.982
		ImpA*	151.8	2260.5	26	746.680
		ImpA*G	138.1	2083.2	24	687.420
		ImpA*ACO	123.6	2119.0	22	691.754
Bộ dữ liệu 4.2h (30 × 30)	50% (450/900)	A*	180	4590	65	1468.505
		ImpA*	175.2	3340.7	43	1085.158
		ImpA*G	177.8	3231.5	42	1053.156

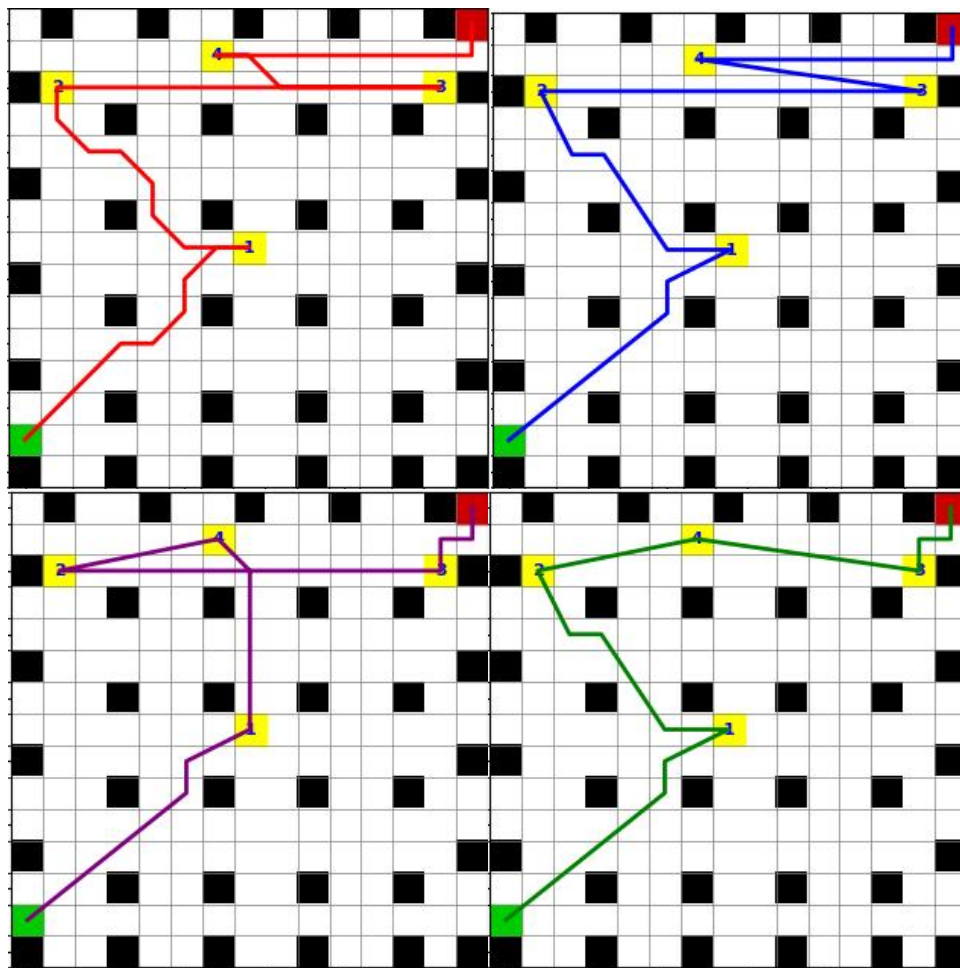
		ImpA*ACO	156.5	3046.4	38	987.932
		A*	168	5850	71	1843.488
Bộ dữ liệu 4.2i (30 × 30)	70% (630/900)	ImpA*	166.4	5318.8	63	1681.117
		ImpA*G	183.5	5324.3	65	1690.157
		ImpA*ACO	154.7	4428.1	53	1406.217
		A*	185	4230	42	1355.600
Bộ dữ liệu 4.2j (30 × 30)	75% (675/900)	ImpA*	185	4230	43	1355.900
		ImpA*G	181	4050	41	1299.700
		ImpA*ACO	171	3870	39	1241.100

Nhìn chung, khi mật độ chướng ngại vật tăng, tất cả các thuật toán đều bị ảnh hưởng đáng kể về chất lượng đường đi và chi phí tìm kiếm.

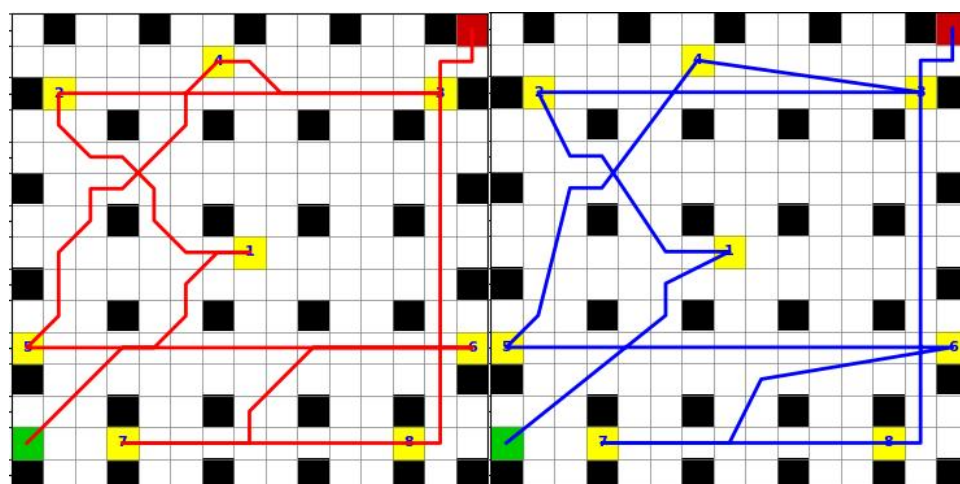
- Với độ dài đường đi, có thể quan sát rõ ràng xu hướng tăng theo mật độ vật cản. Ở mức mật độ thấp (1%–5%), các thuật toán có thể tìm được lối đi ngắn, ít vòng tránh. Tuy nhiên, từ mức 20% trở lên, đường đi bắt đầu phải lách qua các vật cản, dẫn đến quãng đường dài hơn đáng kể.
- Về tổng số góc quay và số lần rẽ, sự gia tăng mật độ cũng kéo theo mức độ gấp khúc cao hơn. A* luôn là thuật toán có tổng góc quay và số lần rẽ lớn nhất. Ngược lại, ImpA* và các phiên bản cải tiến kiểm soát tốt hơn về hướng di chuyển, giúp hành trình mượt và ít gấp khúc hơn đáng kể, kể cả trong bản đồ phức tạp.
- Đối với chỉ số fitness tổng hợp, kết quả cho thấy hiệu suất của các thuật toán giảm dần khi mật độ vật cản tăng. Điều này phản ánh thực tế rằng khi môi trường trở nên chật hẹp, khả năng lựa chọn bị giới hạn, và lợi thế của các chiến lược chọn đích tối ưu cũng bị giảm bớt.

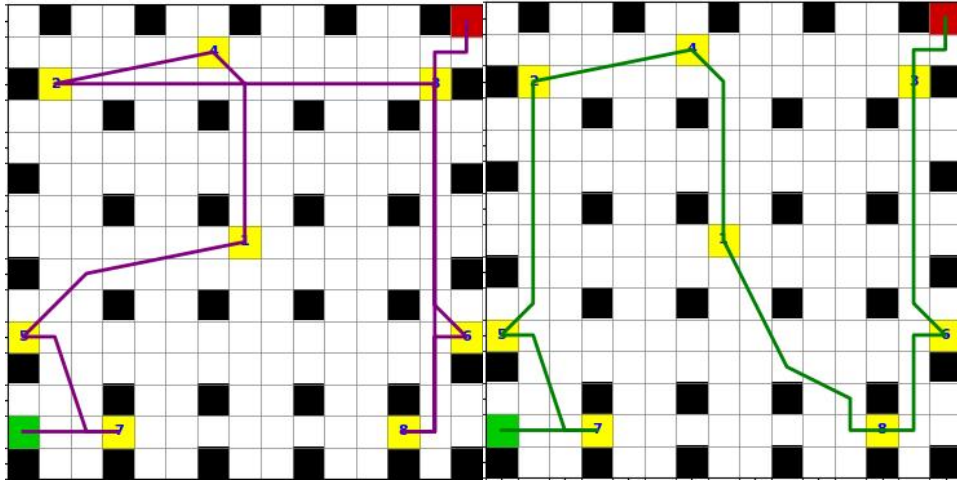
Từ đó, ta có thể thấy mật độ chướng ngại vật có ảnh hưởng rõ rệt đến hiệu quả của các thuật toán. Khi mật độ tăng, quãng đường, số lần rẽ và góc quay đều có xu hướng tăng lên.

4.4.3 Case 3: Ảnh hưởng của số lượng đích đến

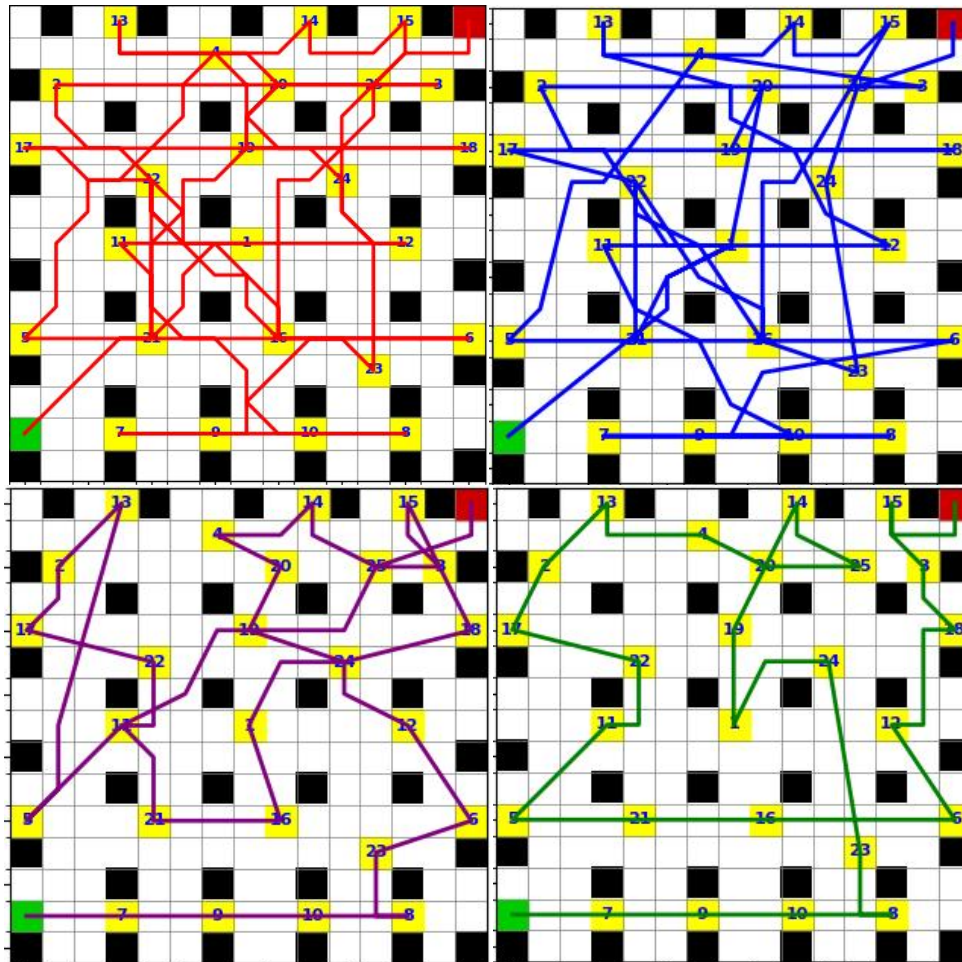


Bộ dữ liệu 4.3a





Bộ dữ liệu 4.3c



Bộ dữ liệu 4.3j

Hình 4.3. Kết quả thực nghiệm case 3

Bảng 4.4. Kết quả thực nghiệm case 3

Môi trường	Mục tiêu	Thuật toán	Độ dài đường đi (m)	Tổng số góc quay (°)	Số lần rẽ	Fitness
Bộ dữ liệu 4.3a (15× 15)	4	A*	47.7	1305	18	415.991
		ImpA*	46.6	994.6	10	319.992
		ImpA*G	36.2	718.2	9	232.624
		ImpA*ACO	33.7	847.1	11	270.887
Bộ dữ liệu 4.3b (15× 15)	6	A*	76.2	1935	29	619.685
		ImpA*	74.5	1449	17	469.607
		ImpA*G	55.7	1192.4	15	384.487
		ImpA*ACO	44.1	1016.1	14	326.652
Bộ dữ liệu 4.3c (15× 15)	8	A*	101.6	2385	33	766.051
		ImpA*	99.4	1827	21	594.152
		ImpA*G	68	1740.5	19	555.060
		ImpA*ACO	53.2	1440	19	458.985
Bộ dữ liệu 4.3d (15× 15)	10	A*	112.5	2925	38	933.882
		ImpA*	109.7	2291.2	25	738.761
		ImpA*G	64.9	1080	16	354.772
		ImpA*ACO	56	1523.9	21	485.859
Bộ dữ liệu 4.3e (15× 15)	12	A*	123.3	3375	44	1075.014
		ImpA*	120.4	2567.6	28	826.813
		ImpA*G	57.4	1006.3	14	329.051
		ImpA*ACO	57.4	1006.3	14	329.051
Bộ dữ liệu 4.3f (15× 15)	15	A*	143.8	4635	60	1466.008
		ImpA*	140	3640.1	39	1159.738
		ImpA*G	62.2	1332.6	22	528.223

		ImpA*ACO	62.2	1332.6	22	528.223
Bộ dữ liệu 4.3g (15× 15)	16	A*	166.3	5265	71	1667.302
		ImpA*	161.6	4075.9	44	1300.602
		ImpA*G	67.9	1986.4	23	629.977
		ImpA*ACO	65.9	1912.6	23	607.055
Bộ dữ liệu 4.3h (15× 15)	20	A*	194.5	5985	80	1897.299
		ImpA*	209.1	5076.3	54	1622.741
		ImpA*G	91.6	2526.4	31	803.844
		ImpA*ACO	81.6	2706.4	29	853.241
Bộ dữ liệu 4.3i (15× 15)	22	A*	215.6	6795	91	2152.027
		ImpA*	166.4	5318.8	63	1681.117
		ImpA*G	104.7	2851.3	38	908.659
		ImpA*ACO	88.7	3053.6	38	962.968
Bộ dữ liệu 4.3j (15× 15)	25	A*	227.2	7335	98	2320.790
		ImpA*	220	5406.8	58	1727.454
		ImpA*G	109.3	3227.6	41	1024.281
		ImpA*ACO	96.2	3443	39	1083.097

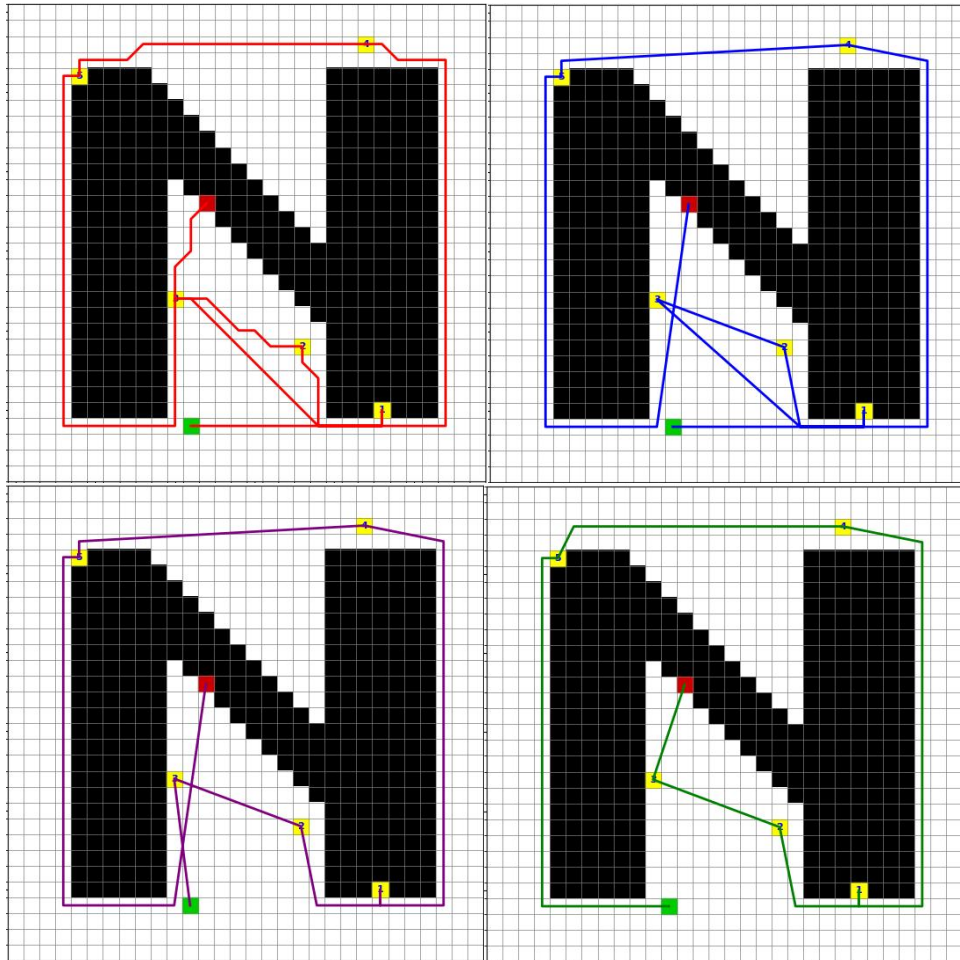
Kết quả cho thấy, khi số lượng đích đến tăng, tất cả các thuật toán đều bị ảnh hưởng rõ rệt về độ dài đường đi, tổng số góc quay, số lần rẽ và chỉ số fitness.

- Với thuật toán A*, độ dài đường đi và các chỉ số khác tăng theo cấp số gần tuyến tính, phản ánh rõ ràng việc không tối ưu được thứ tự ghé đích.
- ImpA*G và ImpA*ACO thể hiện rõ ưu thế về hiệu quả tổng thể.

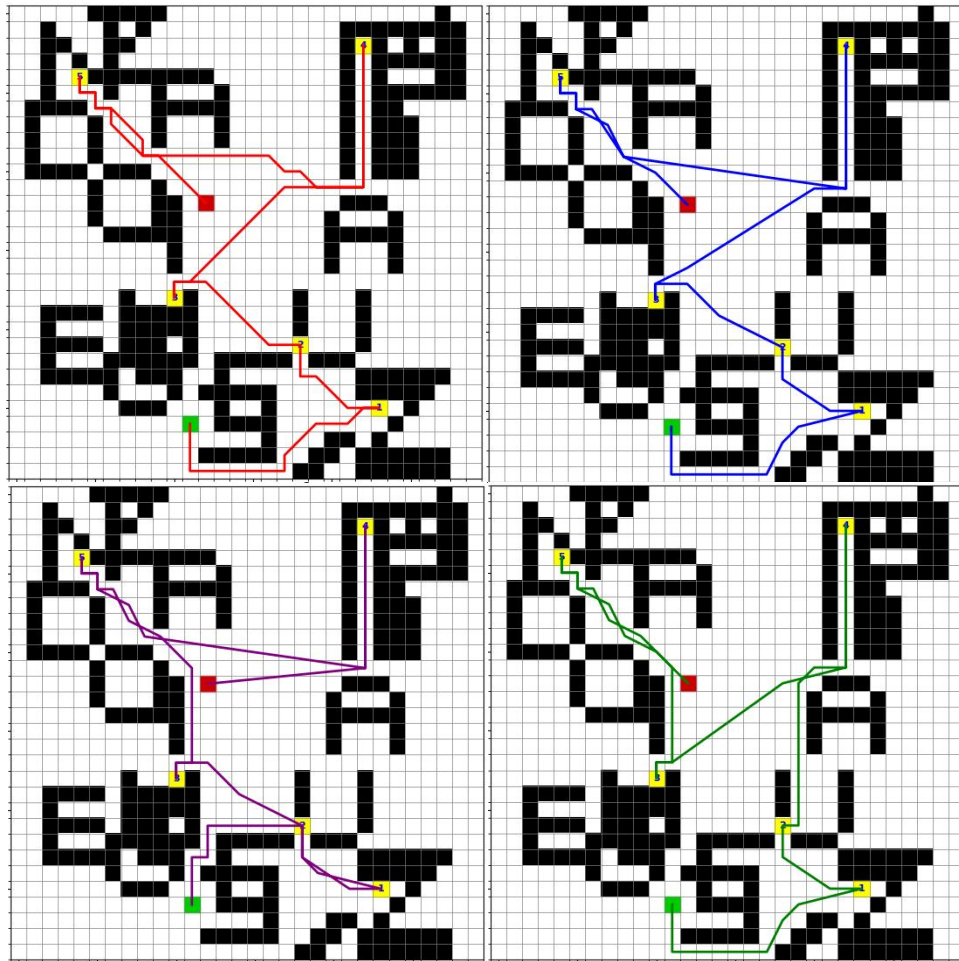
Khi số lượng đích đến tăng, độ dài và độ phức tạp của hành trình cũng tăng theo, ảnh hưởng rõ rệt đến kết quả của các thuật toán. A* bị suy giảm hiệu quả nhanh nhất do không có cơ chế tối ưu thứ tự đích. ImpA* cải thiện hình học hành trình nhưng vẫn

gặp giới hạn ở bài toán đa đích lớn. ImpA*G và ImpA*ACO giúp tối ưu thứ tự ghé đích và cho kết quả tốt nhất ở mức mục tiêu trung bình, tuy nhiên hiệu quả giảm dần khi số lượng đích quá cao. Điều này phản ánh bản chất đánh đổi giữa chi phí tính toán và khả năng khai thác trong môi trường đa mục tiêu.

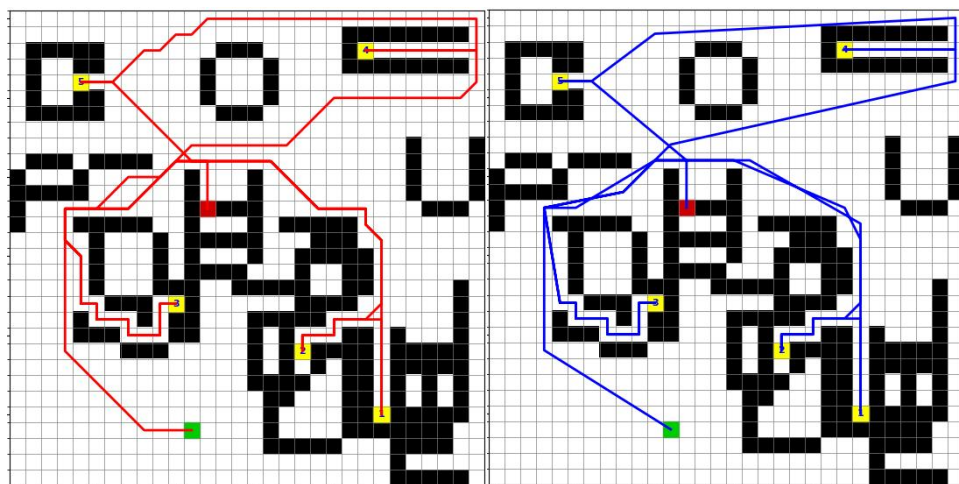
4.4.4 Case 4: Ảnh hưởng của hình dạng chướng ngại vật

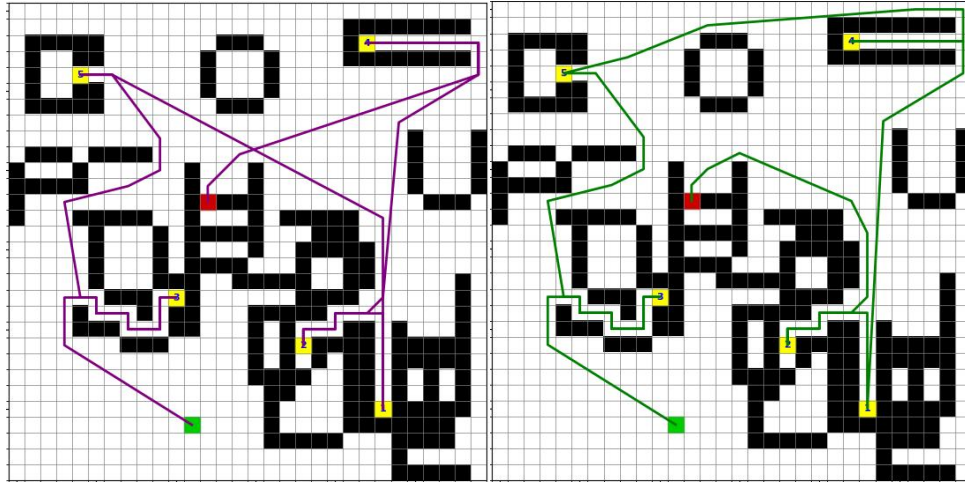


Bộ dữ liệu 4.4d



Bộ dữ liệu 4.4f





Bộ dữ liệu 4.4g

Hình 4.4. Kết quả thực nghiệm case 4

Bảng 4.5. Kết quả thực nghiệm case 4

Môi trường	Thuật toán	Độ dài đường đi (m)	Tổng số góc quay (°)	Số lần rẽ	Fitness
Bộ dữ liệu 4.4a (30×30)	A*	85.9	1260	20	418.348
	ImpA*	82.9	689.7	8	242.471
	ImpA*G	82.8	675	8	238.019
	ImpA*ACO	82.8	675	8	238.019
Bộ dữ liệu 4.4b (30×30)	A*	124.7	1620	24	543.079
	ImpA*	120.8	1043	12	364.829
	ImpA*G	97.6	790.6	10	279.206
	ImpA*ACO	97.6	790.6	10	279.206
Bộ dữ liệu 4.4c (30×30)	A*	110.3	2250	37	730.202
	ImpA*	106.1	1046.5	13	360.295
	ImpA*G	78	911.2	14	308.757
	ImpA*ACO	70.8	712.8	9	244.854

	A*	145.6	2025	28	674.151
Bộ dữ liệu 4.4d	ImpA*	143	1319.3	15	457.456
(30× 30)	ImpA*G	123	1236.4	14	424.302
	ImpA*ACO	108.3	1071.6	13	368.693
	A*	119.4	2070	35	679.271
Bộ dữ liệu 4.4e	ImpA*	116	1389.4	22	469.818
(30× 30)	ImpA*G	95.5	1197.3	20	403.369
	ImpA*ACO	95.5	1197.3	20	403.369
	A*	104.9	3015	39	958.176
Bộ dữ liệu 4.5f	ImpA*	101.6	2214.8	30	714.087
(30× 30)	ImpA*G	106.4	2580.6	32	826.333
	ImpA*ACO	102.4	2531.1	34	810.487
	A*	222.2	5220	71	1676.162
Bộ dữ liệu 4.5g	ImpA*	216.3	4083.4	51	1326.831
(30× 30)	ImpA*G	163	3700.7	43	1188.337
	ImpA*ACO	161.2	3757.2	47	1205.729
	A*	139.2	3420	52	1097.274
Bộ dữ liệu 4.5h	ImpA*	133.3	2100.5	25	690.970
(30× 30)	ImpA*G	133.3	2100.5	25	690.970
	ImpA*ACO	124.3	1480.8	19	499.648
	A*	209.1	3600	59	1181.333
Bộ dữ liệu 4.5i	ImpA*	203.1	2082.1	28	714.269
(30× 30)	ImpA*G	203.7	2253.8	32	767.205
	ImpA*ACO	143.1	1783.7	23	599.249
Bộ dữ liệu	A*	241.8	5490	74	1765.908

4.5j (30× 30)	ImpA*	236.5	4404.6	55	1432.480
	ImpA*G	237.3	4559.4	56	1479.551
	ImpA*ACO	164	3407.6	42	1100.455

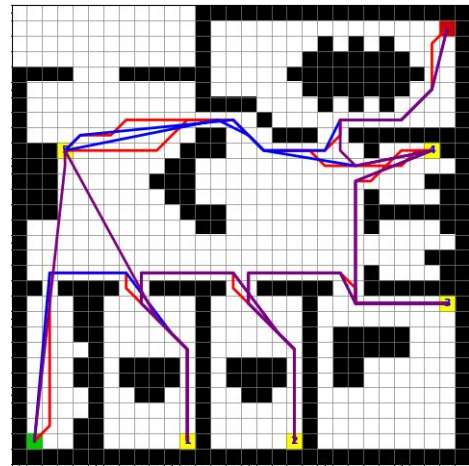
Kết quả từ bảng 4.4 cho thấy các thuật toán cải tiến như ImpA*, ImpAG và ImpA*CO hoạt động tốt và ổn định trong các bản đồ có cùng kích thước, mật độ vật cản và vị trí đích, nhưng khác nhau về hình dạng chướng ngại vật. Tất cả các thuật toán đều tìm được đường đi hợp lệ (tỷ lệ thành công 100%), tuy nhiên chất lượng hành trình và chi phí tìm kiếm bị ảnh hưởng đáng kể bởi cấu trúc hình học của vật cản.

Các bản đồ có vật cản đơn giản như hình chữ C hoặc L thường cho kết quả tốt nhất. Đường đi trong những bản đồ này ngắn hơn, số lần rẽ ít hơn, và thời gian chạy thấp do robot dễ dàng định hướng và thoát khỏi khu vực chướng ngại. Ngược lại, các bản đồ có vật cản dạng chữ O, X hoặc cấu trúc khép kín khiến đường đi phức tạp hơn, dễ bị dẫn vào bẫy hoặc phải vòng tránh, dẫn đến độ dài hành trình và chỉ số fitness tăng đáng kể.

Từ kết quả thực nghiệm có thể thấy rõ: hình dạng chướng ngại vật ảnh hưởng mạnh đến hiệu quả tìm đường. Khi hình dạng càng phức tạp, robot cần tránh né kỹ hơn, khiến độ dài đường đi tăng lên, số lần rẽ nhánh cũng nhiều hơn.

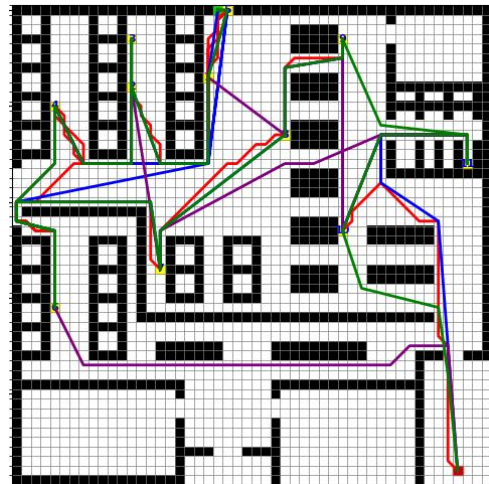
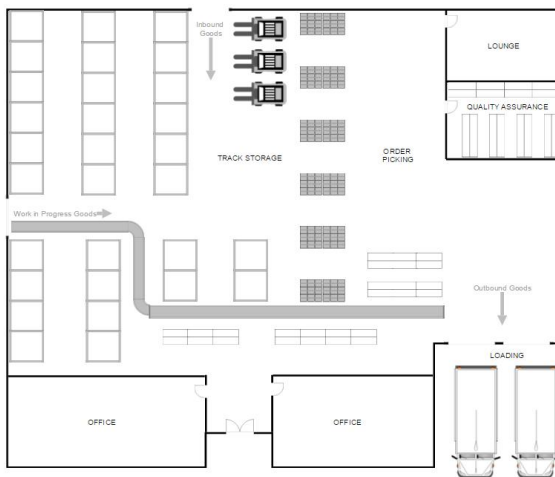
4.4.5 Case 5: Đánh giá dựa trên mô phỏng môi trường thực tế

- Mô phỏng một văn phòng được chia thành nhiều phòng và khu vực chức năng riêng biệt, sử dụng làm môi trường kiểm thử cho các thuật toán lập kế hoạch đường đi.



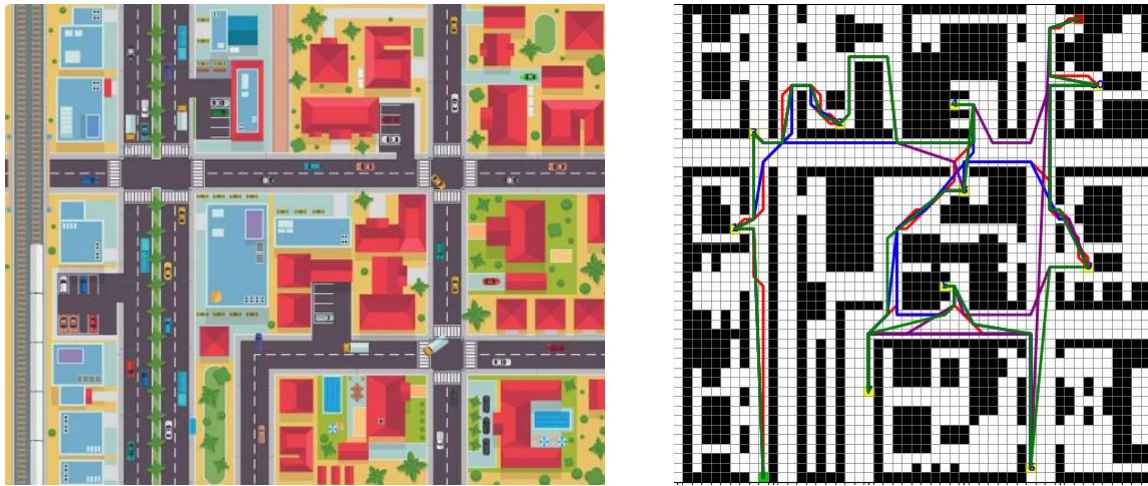
Hình 4.5. Bộ dữ liệu văn phòng.

- Môi trường thử nghiệm mô phỏng sơ đồ một nhà kho công nghiệp có bố trí thực tế gồm nhiều khu vực như lưu trữ hàng, chọn đơn hàng, kiểm định chất lượng, các kệ.



Hình 4.6. Bộ dữ liệu nhà kho.

- Mô phỏng cấu trúc giao thông, đường phố, nhà cửa, cây xanh. Phù hợp để dùng làm môi trường thử nghiệm cho các thuật toán điều hướng đường đi trong đô thị.



Hình 4.7. Bộ dữ liệu thành phố

Bảng 4.6. Kết quả thực nghiệm case 5

Môi trường	Thuật toán	Độ dài đường đi (m)	Tổng số góc quay (°)	Số lần rẽ	Fitness
Bộ dữ liệu 4.5.1 (30× 30)	A*	165.3	3150	47	1025.202
	ImpA*	161.5	2248.7	33	749.096
	ImpA*G	136.5	1947.2	26	646.551
	ImpA*ACO	136.5	1947.2	26	646.551
Bộ dữ liệu 4.5.2 (50× 50)	A*	318.4	5175.0	74	1702.059
	ImpA*	309.5	3236.3	34	1104.899
	ImpA*G	250.8	2540.2	31	871.673
	ImpA*ACO	233.4	2777.9	34	936.914
Bộ dữ liệu 4.5.3 (50× 50)	A*	293	5490	77	1787.293
	ImpA*	287.2	4363.1	51	1439.134
	ImpA*G	270.2	3695.4	41	1228.997
	ImpA*ACO	236.6	3594.1	41	1185.177

Từ bảng kết quả thực nghiệm thu được, có thể nhận thấy rằng các thuật toán cải tiến đều cho hiệu quả tốt hơn đáng kể so với thuật toán A* truyền thống, cả về độ dài

đường đi, số lần rẽ, tổng góc quay lần giá trị fitness tổng hợp. Trong ba bộ dữ liệu mô phỏng – đại diện cho các môi trường có quy mô và bố cục khác nhau – thuật toán A* luôn cho kết quả kém nhất với đường đi dài hơn, số lần rẽ và tổng góc quay lớn, dẫn đến chi phí vận hành cao.

Ngược lại, các thuật toán cải tiến như ImpA*, ImpA*G và ImpA*ACO đều giúp rút ngắn quãng đường, giảm số lần đổi hướng và góc quay, từ đó tối ưu đáng kể giá trị fitness. Đáng chú ý, ImpA*G và ImpA*ACO thường đạt kết quả vượt trội, đặc biệt trong các môi trường lớn.

- *Nhận xét chung:*

Tổng thể, các kết quả cho thấy việc tích hợp các chiến lược cải tiến như heuristics tham lam hoặc tối ưu bầy đàn kiến (ACO) đã giúp thuật toán thích ứng tốt hơn với môi trường thực tế. Trong đó, ImpA*G có xu hướng hiệu quả hơn trong môi trường nhỏ–vừa, còn ImpA*ACO phát huy tốt ở môi trường lớn và phức tạp, nhờ khả năng tối ưu toàn cục tốt hơn. Những kết quả này khẳng định tính khả thi cao của các phương pháp đề xuất trong việc ứng dụng cho các hệ thống robot di động, đặc biệt trong các bài toán yêu cầu tối ưu hóa lộ trình trong không gian nhiều giới hạn. Những kết quả này không chỉ củng cố cơ sở lý thuyết đã trình bày, mà còn mở ra hướng ứng dụng thực tiễn đầy tiềm năng trong các hệ thống robot di động, nơi mà bài toán tối ưu hóa lộ trình là yếu tố then chốt để nâng cao hiệu suất hoạt động, tiết kiệm năng lượng và đảm bảo an toàn vận hành.

KẾT LUẬN

Trong bối cảnh tự động hóa và công nghiệp thông minh không ngừng phát triển, việc xây dựng các chiến lược lập kế hoạch đường đi tối ưu cho robot di động đã trở thành một bài toán quan trọng, mang ý nghĩa cả về lý thuyết lẫn ứng dụng thực tiễn. Đặc biệt, trong môi trường có nhiều chướng ngại vật và mục tiêu di chuyển, yêu cầu đặt ra không chỉ dừng lại ở việc tìm được một đường đi khả thi, mà còn phải đảm bảo tính tối ưu về độ ngắn, độ trơn mượt, khả năng thích ứng và hiệu quả tính toán.

Đề án đã đề xuất ba hướng tiếp cận chính, được phát triển dựa trên nền thuật toán A* truyền thống:

- 1) Thuật toán A* cải tiến (ImpA*) được thiết kế lại với hàm đánh giá thông minh hơn, mở rộng hướng di chuyển theo định hướng mục tiêu và áp dụng kỹ thuật làm mượt quỹ đạo. Biến thể này đặc biệt hiệu quả trong các bài toán đơn mục tiêu, môi trường tĩnh, nơi độ trơn mượt và độ ngắn của đường đi được ưu tiên hàng đầu.
- 2) Biến thể ImpA*G kết hợp giữa A* cải tiến và chiến lược tham lam, giúp lựa chọn mục tiêu gần nhất tại mỗi bước đi. Phương pháp này phù hợp với các môi trường có số lượng đích vừa phải, yêu cầu phản hồi nhanh và thay đổi tác vụ linh hoạt – chẳng hạn như robot giao hàng, robot vệ sinh hoặc điều hướng trong kho hàng vừa và nhỏ. Tốc độ xử lý nhanh và dễ tích hợp là lợi thế lớn, tuy nhiên hành trình tạo ra không đảm bảo tối ưu toàn cục.
- 3) Biến thể ImpA*ACO tích hợp ImpA* với thuật toán tối ưu hóa đàn kiến để xác định thứ tự ghé đích hợp lý trước khi tìm đường chi tiết. Biến thể này thể hiện hiệu quả vượt trội trong các môi trường phức tạp, nhiều đích đến như kho hàng lớn, mạng giao thông đô thị hoặc dây chuyền sản xuất tự động. Tuy nhiên, chi phí tính toán cao khiến nó phù hợp hơn với các hệ thống có thể ưu tiên độ tối ưu hơn thời gian xử lý.

Thực nghiệm trên nhiều kịch bản khác nhau – từ bản đồ giả lập đến mô phỏng môi trường thực tế – cho thấy cả ba biến thể đều có khả năng cải thiện đáng kể các tiêu chí quan trọng: tổng độ dài đường đi, số lần rẽ, tổng góc quay và chỉ số fitness tổng hợp. Tuy nhiên, không có thuật toán nào là tối ưu tuyệt đối cho mọi tình huống. Mỗi biến

thể sẽ phù hợp với một nhóm bài toán và môi trường cụ thể, phụ thuộc vào mục tiêu tối ưu và giới hạn hệ thống.

Tuy đạt được nhiều kết quả khả quan, đề án vẫn còn một số giới hạn nhất định. Các thuật toán được xây dựng chủ yếu trong môi trường 2D, tĩnh, chưa xét đến các yếu tố như chướng ngại vật động, giới hạn động học của robot. Vì vậy, một số định hướng mở rộng có thể được đề xuất như sau:

- Phát triển các phiên bản mở rộng có khả năng thích nghi với môi trường động và không gian 3D.
- Tích hợp các ràng buộc vật lý thực tế như kích thước robot, bán kính quay, mô hình động học.
- Nghiên cứu các giải pháp học thích nghi, chẳng hạn như học tăng cường nhằm tối ưu hóa lựa chọn chiến lược theo thời gian thực và giảm sự phụ thuộc vào thiết lập thủ công.

TÀI LIỆU THAM KHẢO

- [1] U. Nehmzow, *Mobile Robotics: A Practical Introduction*, 2003.
- [2] M. e. a. RUSSO, "A survey on mechanical solutions for hybrid mobile robots," *Robotics*, vol. 9(2), p. 32, 2020.
- [3] L. e. a. Liu, "Path planning techniques for mobile robots: Review and prospect," *Expert Systems with Applications*, vol. 227, p. 120254, 2023.
- [4] H.-y. W.-m. L. e. a. Zhang, "Path planning for the mobile robot: A review," *Symmetry*, vol. 10.10, p. 450, 2018.
- [5] Wikipedia, "A* search algorithm," 20 June 2025. [Online]. Available: https://en.wikipedia.org/wiki/A*_search_algorithm.
- [6] "Greedy Algorithm Tutorial," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/introduction-to-greedy-algorithm-data-structures-and-algorithm-tutorials/>. [Accessed 2025].
- [7] M. a. C. B. Dorigo, "Ant colony optimization theory: A survey," *Theoretical computer science*, Vols. 344(2-3), pp. 243-278, 2005.
- [8] A. e. a. Bakdi, "Optimal path planning and execution for mobile robots using genetic algorithm and adaptive fuzzy-logic control," *Robot. Auton. Syst*, vol. 89, p. 95–109, 2017.
- [9] J. a. Y. S. Han, "Mobile robot path planning with surrounding point set and path improvement," *Appl. Soft Comput*, vol. 57, p. 35–47, 2017.
- [10] M. e. a. Davoodi, "Multi-objective path planning in discrete space," *Appl. Soft Comput*, vol. 13, p. 709–720, 2013.
- [11] D. e. a. Xiang, "Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot," *Scientific reports*, vol. 12.1, p. 13273, 2022.
- [12] P. H. & N. D. T. Luân, "Nghiên cứu ứng dụng thuật toán Aco (ANT COLONY OPTIMIZATION) tối ưu thời gian và chi phí cho dự án xây dựng," *Tạp chí Phát triển KH&CN*, 2010.
- [13] Y. Zhang, D.-W. Gong and J.-H. Zhang, "Robot path planning in uncertain environment using multi-objective particle swarm optimization," *Neurocomputing*, vol. 103, p. 172–185, 2013.
- [14] A. V. S. M. & S. A. Rezvanian, "An overview of ant colony optimization algorithms for dynamic optimization problems," *Optimization Algorithms- Classics and Recent Advances*, 2023.
- [15] Q. e. a. Song, "Intelligent Optimization Algorithm-Based Path Planning for a Mobile Robot," *Computational intelligence and neuroscience*, vol. 2021(1), p. 8025730, 2021.
- [16] H. e. a. Qin, "Review of autonomous path planning algorithms for mobile robots," *Drones*, vol. 7(3), p. 211, 2023.
- [17] C. I. e. a. Okonta, "A heuristic based ant colony optimization algorithm for energy efficient smart homes," *Clean Energy*, 2016.
- [18] S. e. a. Lin, "An intelligence-based hybrid PSO-SA for mobile robot path

- planning in warehouse," *Journal of Computational Science*, vol. 67, p. 101938, 2023.
- [19] F. e. a. Duchoň, "Path Planning with Modified a Star Algorithm for a Mobile Robot," *Procedia engineering*, vol. 96, pp. 59-69, 2014.
- [20] N. e. a. Adzhar, "A review on autonomous mobile robot path planning algorithms.," *Advances in Science, Technology and Engineering Systems Journal*, vol. 5(3), pp. 236-240, 2020.
- [21] A. e. a. Abdulsaheb, "Classical and heuristic approaches for mobile robot path planning: A survey," *Robotics*, vol. 12(4), p. 93, 2023.